

Sample Chapter: Core JDO ISBN 0-13-140731-7

The chapters of the book titled “Core JDO” are being made available for the purposes of review to allow the community to participate in the review and editing of the chapters of the book.

Copyright (c) 2002 Sun Microsystems, Inc. All rights reserved.

FOR PRIVATE USE ONLY: This material is the property of Sun Microsystems, Inc. and is not to be sold, reproduced in any form by any means, or generally distributed without the prior written authorization of Sun Microsystems Inc and Prentice Hall.

To add your comments

- Simply send your uninhibited comments and feedback to the authors at jdoobook@yahoogroups.com
- Or click and add a note using Acrobat where needed. E-mail the PDF back to the authors at jdoobook@yahoogroups.com
- If you would like to receive the chapters in an alternate format, have any questions, concerns or please contact the authors at jdoobook@yahoogroups.com

If debugging is the art of removing bugs, then programming must be the art of inserting them.-Unknown

Developing With JDO

Becoming proficient with using JDO to build applications requires a thorough understanding of the JDO concepts as well as all the classes and interfaces defined in the “javax.jdo” and “javax.jdo.spi” packages. Lets explore these concepts in greater detail and look at all the classes and APIs defined by the JDO specification.

The topics this chapter covers are:

- JDO concepts
- The names, roles and relationships for all the JDO interfaces and classes
- Explanation for each method defined by the PersistenceManagerFactory, PersistenceManager, Transaction, Extent, Query and InstanceCallbacks interfaces
- An overview of the JDO exception classes
- Explanation for each method defined by the JDOHelper, I18NHelper classes
- An overview of the classes and interfaces in the service provider interface package

The examples for this chapter are located in the directory “com/corejdo/examples/chapter5” on the CD accompanying the book and use the persistence capable classes contained in the “com/corejdo/examples/model” directory. See appendix X for a description of these classes.

5.1 JDO CONCEPTS

Being proficient at using JDO to build applications requires a good understanding of the concepts underlying JDO. Some of these have already been introduced in previous chapters to varying degrees and others will be dealt with in greater detail in the chapters that follow.

5.1.1 Persistence Capable

The concept of a persistence capable class was explored in detail in chapter 3. It's the basic mechanism that enables JDO to transparently persist instances of a Java class in a datastore. Any Java class that implements the JDO `PersistenceCapable` interface and adheres to the JDO programming style is said to be persistence capable and can therefore be used with any JDO implementation without recompilation.

Typically a developer would use vendor-supplied tooling to either generate a persistence capable class from scratch or take an existing Java class and make it persistence capable (through source code or byte code manipulation).

5.1.2 JDO Metadata

As introduced in chapter 3 JDO requires additional metadata to be defined for each persistence capable class. This metadata denotes that a given Java class is actually persistence capable, whether it has a persistence capable superclass and the how the class and fields of the class should be treated by the JDO implementation.

The XML file containing the metadata file should contain:

- A header, this specifies the location of the JDO Metadata XML DTD. The location can be the URL as published by SUN Microsystems or it can be some DTD file stored elsewhere:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE jdo SYSTEM "http://java.sun.com/dtd/jdo_1_0.dtd">
```

- A `<jdo>` element, within which the metadata for the persistence capable classes is contained:

```
<jdo>
</jdo>
```

- The `<jdo>` element should contain one or more `<package>` elements, each having a "name" attribute that specifies the name of the package. A `<package>` ele-

ment contains the metadata for the persistence capable classes contained in that package:

```
<package name="com.corejdo.examples.model">
</package>
```

- A <package> element should contain one or more <class> elements, each having a “name” attribute that specifies the name of the Java class. A <class> element can have a number of additional attributes: “identity-type”; “objectid-class”; “requires-extent” and “persistence-capable-superclass” that define how the persistence capable class should be treated by the JDO implementation:

```
<class name="Author">
</class>
```

- A <class> element can contain one or more <field> elements, each having a “name” attribute that specifies the name of the field. A <field> element can have a number of additional attributes: “persistence-modifier”; “primary-key”; “default-fetch-group” and “embedded” that define how the field should be treated by the JDO runtime:

```
<field name="books">
</field>
```

- A <field> element can contain a <collection>, <map> or <array> element depending on the type of the field. These elements specify additional metadata about the field (such as the class of the instances in a collection):

```
<collection element-type="com.corejdo.examples.model.Book">
</collection>
```

JDO defines various defaults for the optional attributes that avoids the need to specify them explicitly. This keeps the average metadata file short and simple. In addition to the elements outlined above, the DTD also allows vendors to specify extensions using the <extension> element (which has “vendor-name”, “key” and “value” attributes).

The metadata for a persistence capable class should be located in a metadata file specific to the class or in a file that contains the metadata for all persistence capable classes in a given package hierarchy. A metadata file specific to a single class should be named after the class itself, but with a “.jdo” suffix. For example, the metadata for the Author class could be located in a file called “Author.jdo”. Alternatively the metadata could be located in a file called “package.jdo”. This file would then contain the metadata for the Author class as well as any other persistence capable class in the same package hierarchy. In both situations the metadata file should be available at runtime as a resource that can be loaded by the same

class loader that loaded the class itself (i.e. it should be in the classpath). For example, the metadata for the class “com.corejdo.examples.model.Author” should be contain in one of the following files:

- com/corejdo/examples/model/Author.jdo
- com/corejdo/examples/model/package.jdo
- com/corejdo/examples/package.jdo
- com/corejdo/package.jdo
- com/package.jdo
- system.jdo

As explained in chapter 3, the initial JDO 1.0 specification didn’t actually mandate the location and name of the XML metadata files. To aid in portability the 1.0.1 maintenance revision of the specification changed this to what has just been described.

Prior to this revision the name of a metadata file that contained multiple classes was the package name itself with a “.jdo” suffix and it would have been located in the directory that contained the package directory. In the Author example it would have been called “model.jdo” and would have been located in the “com/corejdo/examples/” directory. Some JDO implementations may still use this naming convention.

Appendix X provides greater details on the exact syntax of the JDO metadata XML file and the purpose of each element and attribute.

5.1.3 Default Fetch Group

When a persistent object is retrieved from the datastore the JDO implementation initially retrieves the values of those fields that are part of what is referred to as the “default fetch group” for the class. If a persistent object has been retrieved from the datastore, then as a minimum the values of all the fields that are part of the default fetch group will be in-memory. This concept allows the JDO implementation to optimize access to the fields in the default fetch group since it doesn’t have to mediate access to each field individually once a persistent object has been retrieved. For fields not in the default fetch group the JDO implementation mediates each access ensuring that the field has first been retrieved from the datastore.

By default, fields of primitive types, java.util.Date and supported system classes from java.lang and java.math packages are automatically part of the default fetch group. A field can be explicitly added to the default fetch group by setting the “default-fetch-group” attribute to “true” in the metadata for the field. If a field that references another persistent object is made part of the default fetch group then effectively the JDO implementation will retrieve the referenced persistent object also.

5.1.4 Persistence by Reachability

Chapter 3 introduced the concept of persistence by reachability. It simply means that instances of persistence capable classes or supported system classes that are referenced from an instance that is itself made persistent will implicitly be made persistent also and so on for the entire graph of reachable instances. Likewise any transient instance that is later added to an already persistent instance will be made persistent too.

JDO defines that the persistence by reachability calculation is initially done during `PersistenceManager.makePersistent()` and then redone during `Transaction.commit()`. Therefore a transient instance that is added to a graph after `makePersistent()` will not itself become persistent until `commit()` (or as a result of a later call to `makePersistent()`).

Figure 5.1 shows a graph of instances and identifies those that would be implicitly made persistent.

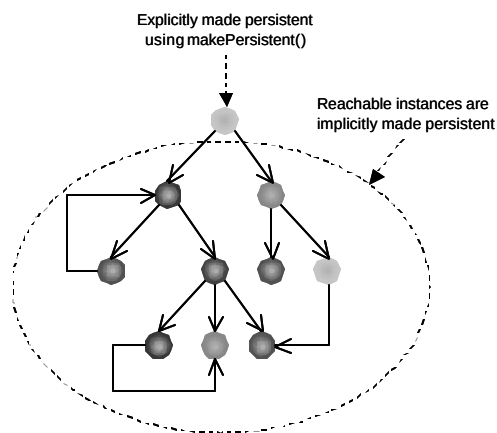


Figure 5-1 Persistence by Reachability

Even if the graph of reachable instances is cyclic in nature, or a particular instance is referenced from multiple others or is already persistent, the JDO implementation will ensure that each is made persistent only once.

Persistence by reachability negates the need for the application to explicitly track all instances it needs to make persistent as it creates them, instead it just needs to make the root instance of a graph persistent and the JDO implementation will implicitly take care of the rest.

5.1.5 First and Second Class Objects

JDO uses the concept of first and second class objects to differentiate how a JDO implementation manages instances of user-defined, persistence capable classes (persistent objects) versus instances of supported system defined classes (like Java collection classes) and arrays.

A first class object is an instance of a persistence capable class and it has a JDO object identity and can be stored in the datastore. A second class object does not have any JDO object identity and can only be stored in the datastore as belonging to a first class object.

As an example, if a persistence capable class referenced a `java.util.HashSet`, then whilst instances of the persistence capable class would be first class objects, the instances of `HashSet` would be second class. This means that instances of supported system classes or arrays are not themselves persisted in the datastore, rather a representation of a given instance is persisted as part of a first class object.

Since second class objects can only be persisted as belonging to a first class object it is therefore not possible for multiple first class objects to reference the same second class object in the datastore. For example, at some point within a transaction multiple in-memory persistent objects may reference the same `java.util.HashSet` instance. However, when the transaction commits and they are persisted in the datastore, each creates its own representation of the `HashSet` instance. When retrieved again into memory, each persistent object will re-create its own `HashSet` instance.

In addition it is not necessary to explicitly delete second class objects. When a first class object is deleted using the `deletePersistent()` method on `PersistenceManager` the JDO implementation automatically ensures that the representation of any associated second class objects are also deleted from the datastore.

5.1.6 Object Identity

Chapter 3 introduced the concept of JDO object identity and the different types of identity that can be used with JDO. Essentially, every persistent object has a unique JDO object identity. An in-memory instance is associated with its representation in the underlying datastore via this identity. Object identity is central to how a JDO implementation manages and persists instances of persistence capable classes.

JDO defines three types of object identity:

- Datastore Identity – a persistent object's identity is automatically generated.
- Application Identity – a persistent object's identity is determined by the values of certain fields (those whose "primary-key" attribute is "true" in the metadata for the class).
- Non-durable Identity – a persistent object's identity is not persistent at all and is only valid whilst in-memory.

By default a persistence capable class is assumed to have datastore identity. The identity type of a persistence capable class can be explicitly specified using the “identity-type” attribute in the class’ metadata. The attribute can be “datastore” (the default), “application” or “nondurable”.

If using application identity then the “objectid-class” attribute is also required to identify the user-defined class used to represent the persistence capable class’ identity.

5.1.7 Object Lifecycle

JDO defines a number of states that an in-memory instance of a persistence capable class can be in. These states are used by the JDO implementation to keep track of whether the fields of a given instance have been retrieved from the underlying datastore within the current transaction, whether any fields have been modified by the application or if the application has deleted it. The transition from state to state is referred to as an object’s lifecycle and the JDO specification denotes which transitions are valid and which aren’t.

As explained in chapter 3, conceptually the basic states are:

- Transient – a non-persistent, normal Java object.
- Persistent – a persistent object whose fields have been retrieved from the datastore in the current transaction.
- Hollow – a persistent object whose fields have not yet been retrieved from the datastore.

The object lifecycle is only of real concern to a JDO implementation; there is nothing a JDO application itself needs to do. However, using the InstanceCallbacks interface or via JDOHelper class an application can interact with the JDO implementation as certain state transitions occur or interrogate the state of a particular instance of a persistence capable class.

See chapter 4 for a comprehensive description of all the states and state transitions that make up a persistent object’s lifecycle.

5.1.8 Transactions

Typically persistent objects must be created, retrieved, modified or deleted within the context of a transaction. The underlying datastore uses transactions to enforce concurrency control between multiple applications and ensure that upon commit, all changes within a transaction get written to the datastore or none of them do (in the case of failure).

JDO supports both unmanaged and managed transaction control. For unmanaged transactions the application itself must explicitly start and end a transaction. For managed transactions, transactions are implicitly started and ended by an external coordinator (ala the Java Transaction Service). Managed transactions are typical if using JDO within the J2EE environment and are covered in more detail in [chapter X](#).

Since different datastores use different approaches to managing concurrency control JDO does not explicitly define the semantics of how concurrency control should be enforced or when concurrency conflicts should be detected. It instead defines the concept of a “datastore transaction” and simply says that a datastore transaction should exhibit ACID properties. See chapter 3, section 3.14.1 for more details on ACID transactions.

5.1.8.1 Non-transactional Read/Write

Basic database principles dictate that a transaction should be short lived to avoid concurrency conflicts between different applications running concurrently. If an application begins a transaction and retrieves persistent objects it can potentially block other applications from accessing or updating those same persistent objects.

For applications that need to access persistent objects over an extended period of time JDO specifies a number of optional features that allow access to persistent objects outside of a normal transaction and therefore avoid potential concurrency issues.

The non-transactional read and write feature allows an application to retrieve persistent objects and update them without first beginning a transaction. The following `PersistenceManagerFactory` options denote if these features are supported by a JDO implementation:

- `javax.jdo.option.NontransactionalRead`
- `javax.jdo.option.NontransactionalWrite`

Persistent objects that are accessed in this way are referred to as being non-transactional.

Since the persistent objects are accessed without using a transaction the underlying datastore is not going to enforce any concurrency control. Once retrieved into memory a persistent object is potentially no longer in sync with the underlying datastore since another application might have modified or even deleted the persistent object after it was retrieved.

An in-memory persistent object can also be modified outside of a transaction, but if the instance is later accessed within a transaction any changes are lost (the persistent object is effectively re-retrieved from the datastore within the current transaction).

Non-transactional support is useful for applications that want to keep persistent objects that are rarely modified in-memory for reference purposes to avoid the overhead of having to keep retrieving them from the datastore within each transaction. However it is the applications responsibility to ensure that the in-memory instances are kept up to date with respect to changes in the underlying datastore.

The methods `setNontransactionalRead()` and `setNontransactionalWrite()` on `Transaction` and `PersistenceManagerFactory` can be used to explicitly enable or disable these features.

5.1.8.2 Optimistic Transactions

Similar to non-transactional read and write this feature allows an application to retrieve and modify persistent objects over a prolonged period of time without requiring an actual datastore transaction for the entire duration. But unlike non-transactional read and write it does ensure transactional consistency. The following `PersistenceManagerFactory` option denotes if this feature is supported by a JDO implementation:

- `javax.jdo.option.Optimistic`

Using an optimistic transaction is similar to using non-transactional read in that persistent objects are retrieved from the datastore and managed in-memory as non-transactional instances outside of an actual datastore transaction. This means that any other application can modify the persistent objects once they have been retrieved, in which case the in-memory instances will no longer be in sync with the actual persistent objects in the datastore. With optimistic transactions however, the JDO implementation ensures that any conflicting updates are detected at commit time.

When a persistent object is first retrieved within an optimistic transaction it becomes a non-transactional instance. If modified, the in-memory instance is marked as being modified and transitions to being transactional, likewise for new or deleted persistent objects. An application can also choose to explicitly make non-transactional instances transactional using the `makeTransactional()` method on `PersistenceManager`.

At commit time all transactional instances are checked against the datastore to ensure that no other application has modified them since they were first retrieved. If there has been a conflicting update `JDOUserException` is thrown and the commit fails. An application can either rollback the transaction and start again or refresh the conflicting instances to resolve the differences and retry the commit.

The method `setOptimistic()` on `Transaction` and `PersistenceManagerFactory` can be used to explicitly enable or disable this feature.

5.1.8.3 Retaining Values

By default, at the end of a transaction all in-memory persistent objects transition to being “hollow” and if needed will be re-retrieved from the datastore within the next transaction. This default behavior can be changed.

The retain values feature allows an application to retain the fields of persistent objects in-memory after a transaction ends. The following `PersistenceManagerFactory` option denotes if this feature is supported:

- `javax.jdo.option.RetainValues`

If using retain values, then at the end of a transaction all in-memory persistent objects transition to being non-transactional rather than hollow. This allows an application to con-

tinue to access the fields of the persistent objects outside of the transaction. However as per the non-transactional read feature, the in-memory persistent objects will no longer be in sync with the underlying datastore.

The method `setRetainValues()` on `Transaction` and `PersistenceManagerFactory` can be used to explicitly enable or disable this feature.

5.1.8.4 Restoring Values

The restore values feature is a mandatory JDO feature and can be used to revert the fields of in-memory persistent objects on rollback to their values as of the beginning of the transaction or the point in time they were made persistent within the transaction.

If used by itself then on rollback the fields of any persistent objects that were made persistent within the transaction will be reverted to their original values and the persistent objects will transition to being transient instances.

If used in conjunction with the `RetainValues` option then on rollback the fields of all in-memory persistent objects will be reverted and they will transition to being non-transactional rather than hollow, as per the behavior for `RetainValues` on commit.

If restore values is not used then the fields of any modified persistent objects retain the modified values, even though these have not been committed to the datastore.

The method `setRestoreValues()` on `Transaction` and `PersistenceManagerFactory` can be used to explicitly enable or disable these options.

5.1.8.5 Transient Transactional

In addition to managing persistent objects transactionally JDO has an optional feature that allows an application to make transient (i.e. non-persistent) instances of persistence capable classes transactional.

The transient transactional feature allows an application to specify that changes to a transient instance of a persistence capable class should be managed transactionally by the JDO implementation. An application can make a transient instance transactional so that changes made to the instance during a transaction will be undone if the transaction rolls back. The following `PersistenceManagerFactory` option denotes if this feature is supported:

- `javax.jdo.option.TransientTransactional`

A transient instance is made transactional using the `make transactional` methods defined on `PersistenceManager`. Any modification of a transient transactional instance during a transaction will be subject to rollback. Any modification made outside of a transaction will not be. A transient transactional instance remains transactional until explicitly made non-transactional using the `make non-transactional` methods defined on `PersistenceManager`.

On rollback a transient transactional instance's fields are reverted to the values they had at the beginning of the current transaction (or the values they had when they were made transactional, if this occurred within the current transaction).

A **PersistenceManager** embodies a connection to a datastore and a cache of in-memory persistent objects that implement the **PersistenceCapable** interface. The **PersistenceManager** interface is the primary means by which an application interacts with the underlying datastore and persistent objects.

From a **PersistenceManager** an application can get one or more **Query** instances. A **Query** is the how an application can find persistent objects by their field values.

In addition to a **Query** an application can get an **Extent** from a **PersistenceManager**. An **Extent** represents all persistent objects of a specified class (and optionally subclasses) in the datastore and can be used as input to a **Query** or iterated through in its own right.

A **Transaction** allows an application to control the transaction boundaries in the underlying datastore. There is a one-to-one correlation between a **Transaction** and a **PersistenceManager** (there can only be one ongoing **Transaction** per **PersistenceManager** instance). **Transactions** must be begun and either committed or aborted.

The **InstanceCallbacks** interface can be implemented by a **PersistenceCapable** class and if implemented the **PersistenceManager** will invoke the relevant callback methods as a persistent object's lifecycle state changes.

JDOHelper can be used to create a **PersistenceManagerFactory** from a set of properties and provides convenience methods to interrogate the state of instances of classes that implement **PersistenceCapable**.

Every persistent object has an associated instance of **StateManager**. This instance manages the persistent fields and lifecycle state of the in-memory persistent object.

JDOImplHelper is used by the JDO implementations to get metadata on **PersistenceCapable** classes; it is not supposed to be used by an application. **JDOImplHelper.Meta** contains the metadata for a given class. To receive notification when a **PersistenceCapable** class is loaded a JDO implementation registers a **RegisterClassListener** with the **JDOImplHelper**. For every **PersistenceCapable** class that is loaded the registered listeners will be invoked using a **RegisterClassEvent**.

JDOPermission contains the access privileges that restrict what can call certain methods on **JDOImplHelper** and **PersistenceCapable** that deal with metadata and modification of the state of a persistent object.

I18NHelper is used by the JDO classes to get messages from a resource bundle. A default resource bundle ("Bundle.properties") is provided in the JDO JAR.

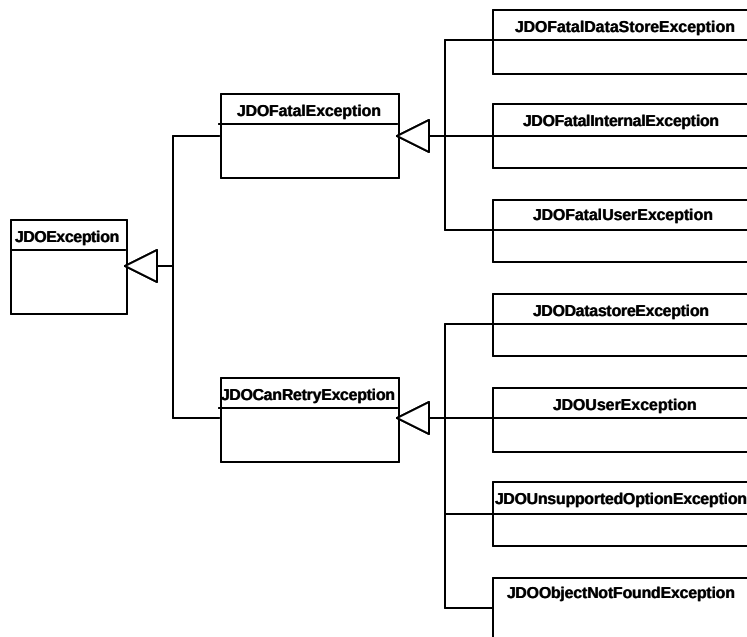


Figure 5-3 JDO Exceptions Class diagram

JDOException is the superclass for all JDO exceptions and extends `java.lang.RuntimeException`. JDO classifies exceptions broadly into fatal or non-fatal exceptions. **JDOFatalException** is the superclass for all fatal exceptions and **JDOCanRetryException** is the superclass for all non-fatal exceptions.

Non-fatal exceptions indicate that an operation failed but can be retried. **JDODataStoreException** indicates a problem with the datastore. **JDOUserException** indicates invalid or incorrect application behavior and **JDOUnsupportedOptionException** indicates an application has tried to use a JDO optional feature that isn't supported by the JDO implementation being used. **JDOObjectNotFoundException** indicates that a persistent object could not be found in the datastore, this exception was added in the JDO 1.0.1 maintenance revision of the specification.

Fatal exceptions indicate that the only recourse is to start again. **JDOFatalDataStoreException** indicates a serious problem with the datastore. **JDOFatalInternalException** indicates a serious problem within the JDO runtime and **JDOFatalUserException** indicates a serious problem caused by incorrect application behavior.

The rest of this chapter provides a detailed overview of the JDO classes and interfaces. The classes and interfaces are grouped as follows:

Basic APIs

PersistenceManagerFactory
PersistenceManager
Transaction
Query
Extent
InstanceCallbacks

Exception Classes

JDOException
JDOCanRetryException
JDODataStoreException
JDOUserException
JDOUnsupportedOptionException
JDOFatalException
JDOFatalDatastoreException
JDOFatalInternalException
JDOFatalUserException

Additional APIs

JDOHelper
I18NHelper

Service Provider APIs

PersistenceCapable
StateManager
JDOImplHelper
JDOPermission

The following classes are used internally by the JDO classes themselves and not covered:

JDOImplHelper.Meta
RegisterClassListener
RegisterClassEvent

5.3 BASIC APIs

The following six interfaces provide the basic set of JDO APIs that most applications will need to use:

- PersistenceManagerFactory
- PersistenceManager

- Transaction
- Query
- Extent
- InstanceCallbacks

In the following sections, for each interface there is a description of its purpose followed by an overview of each method and then examples that show how to use those methods in practice.

5.3.1 javax.jdo.PersistenceManagerFactory

The PersistenceManagerFactory interface is typically the starting point for any JDO application, from it an application gets instances of PersistenceManager. The PersistenceManagerFactory has a number of standard properties that can be configured and in addition vendors can add their own properties to their PersistenceManagerFactory implementations, although all properties should follow the JavaBean pattern of setters and getters.

PersistenceManagerFactory is responsible for managing the physical connections to the underlying datastore and may or may not implement connection pooling. In addition a PersistenceManagerFactory can be used by both managed and non-managed applications. For non-managed applications connection related properties are used to indicate how to make a connection to the datastore. For managed applications connection management is delegated to an external connection factory.

Chapter X provides more details on managed and non-managed applications. Simply put a managed application typically runs within a J2EE container, a non-managed application is a standalone Java application.

A non-managed application creates its own connections to the datastore and explicitly manages its own transactions. A managed application gets connections from a connection factory and may delegate transaction management to the container it runs within.

5.3.1.1 Creating a PersistenceManagerFactory

There are three main ways to create a PersistenceManagerFactory:

JNDI Since PersistenceManagerFactory implements Serializable it can be stored and looked up via JNDI. This is the typical approach for a managed application. The properties of a PersistenceManagerFactory created in this way cannot be modified and the PersistenceManagerFactory instance cannot be serialized again.

JDOHelper The getPersistenceManagerFactory() method creates a PersistenceManagerFactory from a set of properties, one of which specifies the actual class name of the vendor's PersistenceManagerFactory. This is the expected and

portable approach for non-managed applications to get a `PersistenceManagerFactory` instance. JDO requires each vendor to implement a static method on their `PersistenceManagerFactory` called `getPersistenceManagerFactory()`, JDOHelper uses this to create a specific `PersistenceManagerFactory` instance. The properties of a `PersistenceManagerFactory` created in this way cannot be modified and the `PersistenceManagerFactory` instance cannot be serialized.

Constructors Each vendor can provide their own constructors that allow an application to directly construct a `PersistenceManagerFactory` instance. This is a suitable for non-managed applications, but has the disadvantage that it is non-portable and the code would need to be changed per JDO implementation. The properties of a `PersistenceManagerFactory` created in this way can be modified up to the first call to `getPersistenceManager()` and the `PersistenceManagerFactory` instance can be serialized.

The reason a `PersistenceManagerFactory` instance created by either a JNDI lookup or by JDOHelper is not serializable is to avoid a security hole. Since a `PersistenceManagerFactory` can potentially contain a password, if it was serializable it could expose this password.

5.3.1.2 PersistenceManagerFactory Properties

The standard properties defined by JDO for `PersistenceManagerFactory` are split between those that control the creation of connections to the underlying datastore (such as `ConnectionURL` and `ConnectionUserName`) and those that configure the default behavior of `PersistenceManager` instances (such as `IgnoreCache` and `MultiThreaded`).

Depending on whether the application is managed or non-managed different connection based properties apply. Table 5-1 lists the non-managed connection properties:

Table 5-1 Non-managed Connection Properties

Property	Method Summary
<code>ConnectionDriverName</code>	String <code>getConnectionDriverName()</code> void <code>setConnectionDriverName(String name)</code> This property specifies the class name of the driver used when connecting to the datastore. A JDO implementation that uses JDBC might use this property to allow the application to specify the name of the JDBC driver to be used.
<code>ConnectionPassword*</code>	void <code>setConnectionPassword(String password)</code> This property specifies the password of the user used to connect to the datastore.
<code>ConnectionURL</code>	String <code>getConnectionURL()</code> void <code>setConnectionURL(String url)</code> This property specifies the URL used to make the connection and is datastore specific. A JDO implementation that uses JDBC might expect this to be of the form "jdbc:subprotocol:subname". Some implementations may allow the username and password.

Property	Method Summary
	word to be specified in the connection URL.
ConnectionUserName*	String getConnectionUserName() void setConnectionUserName(String name)
	This property specifies the user name used to connect to the datastore.
	*The user name and password can also be specified separately in the call to getPersistenceManager().

The values of these properties are dependent on the underlying datastore being used, not all maybe required or even relevant. The documentation for a given JDO implementation should identify which of these properties are used and what values they should be given.

For managed applications the PersistenceManagerFactory uses an external connection factory to create datastore connections. If any of the connection factory properties are set (non-null) then the un-managed properties above are effectively ignored and the external connection factory is used instead.

Typically a managed application would not explicitly create or configure a PersistenceManagerFactory, it would look one up via JNDI instead. A PersistenceManagerFactory instance would be initialized and registered with JNDI beforehand (although exactly how this is done depends on the managed environment being used and the JDO implementation).

Table 5-2 lists the managed connection properties:

Table 5-2 Managed Connection Properties

Property	Method Summary
ConnectionFactory	Object getConnectionFactory() void setConnectionFactory(Object factory)
	This property specifies the external connection factory used to create connections. The factory interface itself is not defined by JDO and is implementation dependent. If set the PersistenceManagerFactory will not use it's own connection pooling.
	A JDO implementation that uses JDBC might accept an instance of javax.sql.DataSource as a connection factory for JDBC connections.
ConnectionFactoryName	String getConnectionFactoryName() void setConnectionFactoryName(String name)
	This property specifies the JNDI name of the connection factory, if the ConnectionFactory property has not been set then the PersistenceManagerFactory will lookup the connection factory via this name using JNDI.
ConnectionFactory2	Object getConnectionFactory2() void setConnectionFactory2(Object factory)
	This property specifies an alternative external connection factory used to create connections for optimistic transactions. Typically in managed environments transaction control is delegated to an external coordinator, however when using optimistic transactions implicit transaction control is not appropriate.

Property	Method Summary
ConnectionFactory2Name	String getConnectionFactory2Name() void setConnectionFactory2Name(String name)

This property specifies the JNDI name for an alternative connection factory if the ConnectionFactory2 property has not been set then the PersistenceManagerFactory will lookup the connection factory via this name using JNDI.

*A PersistenceManagerFactory that implements its own connection pooling may utilize these properties also, even for a non-managed application.

Table 5-3 lists the configuration properties that provide default values for each PersistenceManager instance created by a given PersistenceManagerFactory:

Table 5-3 Configuration Properties

Property	Method Summary
IgnoreCache	boolean getIgnoreCache() void setIgnoreCache(boolean flag)
	This property specifies whether the PersistenceManager can ignore in-memory changes when creating Extent and Query instances. See section X.X, PersistenceManager for more details.
Multithreaded	boolean getMultithreaded() void setMultithreaded(boolean flag)
	This property specifies whether the PersistenceManager should synchronize itself for multithreaded use. See section X.X, PersistenceManager for more details.
NontransactionRead	boolean getNontransactionalRead() void setNontransactionalRead(boolean flag)
	This property specifies whether the PersistenceManager should allow non-transaction reads by default. See section X.X, PersistenceManager for more details.
NontransactionWrite	boolean getNontransactionalWrite() void setNontransactionalWrite(boolean flag)
	This property specifies whether the PersistenceManager should allow non-transactional writes by default. See section X.X, PersistenceManager for more details.
Optimistic	boolean getOptimistic() void setOptimistic(boolean flag)
	This property specifies whether the PersistenceManager should use optimistic transactions by default. See section X.X, Transaction for more details.
RestoreValues	boolean getRestoreValues() void setRestoreValues(boolean flag)
	This property specifies whether the PersistenceManager should restore the fields of persistent and transactional instances after rollback by default. See section X.X, PersistenceManager for more details.

Property	Method Summary
RetainValues	boolean getRetainValues() void setRetainValues(boolean flag)
This property specifies whether the PersistenceManager should retain the fields of persistent instances after commit by default. See section X.X, PersistenceManager for more details.	

5.3.1.3 Method Summary

In addition to the methods to get and set properties the PersistenceManagerFactory has four other methods:

PersistenceManager getPersistenceManager()

Returns a PersistenceManager instance.

The PersistenceManager will be configured based on the connection and configuration properties of the PersistenceManagerFactory.

After the first call to this method the properties of the PersistenceManagerFactory can no longer be modified.

PersistenceManager getPersistenceManager(String name, String password)

Returns a PersistenceManager instance using the specified user name and password.

The PersistenceManager will be configured based on the connection and configuration properties of the PersistenceManagerFactory, except that the specified user name and password will be used. If the PersistenceManagerFactory implements connection pooling then it will ensure it returns a connection for the specified user.

After the first call to this method the properties of the PersistenceManagerFactory can no longer be modified.

Properties getProperties()

Returns the non-configurable, vendor specific properties of the JDO implementation.

JDO defines two standard vendor properties, "VendorName" and "VersionNumber". All others are implementation specific.

CollectionSupportedOptions()

Returns a collection of strings that represent the optional features and/or query languages that the JDO implementation supports.

Table 5-4 lists the strings used to denote support for the standard JDO optional features and query language:

Table 5-4 Optional Features

Optional Feature	Optional Feature
javax.jdo.option.TransientTransactional	javax.jdo.option.LinkedList
javax.jdo.option.NontransactionalRead	javax.jdo.option.TreeMap
javax.jdo.option.NontransactionalWrite	javax.jdo.option.TreeSet
javax.jdo.option.RetainValues	javax.jdo.option.Vector
javax.jdo.option.Optimistic	javax.jdo.option.Map
javax.jdo.option.ApplicationIdentity	javax.jdo.option.List
javax.jdo.option.DatastoreIdentity	javax.jdo.option.Array
javax.jdo.option.NonDurableIdentity	javax.jdo.option.NullCollection
javax.jdo.option.ArrayList	javax.jdo.option.ChangeApplicationIdentity
javax.jdo.option.Hashtable	javax.jdo.query.JDOQL
javax.jdo.option.HashMap	

5.3.1.4 Usage Guidelines

A non-managed application can create a `PersistenceManagerFactory` via JNDI, `JDOHelper` or explicitly using a vendor-supplied constructor.

If using JNDI then a `PersistenceManagerFactory` needs to be initially created via an implementation specific constructor and bound to its JNDI name. When the application looks up the `PersistenceManagerFactory` via JNDI it will then get a de-serialized instance. Typically a `PersistenceManagerFactory` instance is shared within an application, in which case a singleton pattern should be used to manage the lookup and sharing of the `PersistenceManagerFactory`.

The following code snippet from “`PMFSingletonFromJNDIExample.java`” shows how a `PersistenceManagerFactory` can be created and registered with JNDI and then looked and managed as a singleton instance:

```
import javax.naming.*;
import javax.jdo.*;
import com.versant.judo.PersistenceManagerFactoryImpl;
```

```

public class PMFSingletonFromJNDIExample {

    // JNDI name for the PersistenceManagerFactory instance
    private static String name =
        PMFSingletonFromJNDIExample.class.getName();

    // PersistenceManagerFactory Singleton
    private static PersistenceManagerFactory pmf;

    /*
     * This method returns the singleton
     * PersistenceManagerFactory. If required it first
     * initializes the singleton by looking it up via JNDI.
     * If it can't lookup the PersistenceManagerFactory via
     * JNDI it throws a JDOFatalUserException
     */
    public static PersistenceManagerFactory
    getPersistenceManagerFactory() {

        if (pmf == null) {

            try {

                InitialContext context = new InitialContext();

                pmf = (PersistenceManagerFactory)
                    context.lookup(name);
            }

            catch (NamingException e) {

                throw new JDOFatalUserException(
                    "Can't lookup PersistenceManagerFactory: " +
name,
                    e);
            }

            return pmf;
        }

        /*
         * This class can be run to create a
         * PersistenceManagerFactory instance
         * and register it with JNDI.

```

```

    */
    public static void main(String[] args) {
        if (args.length != 3) {
            System.out.println(
                "Invalid arguments, use: <url> <name> <password>");
            System.exit(-1);
        }

        PersistenceManagerFactory pmf =
            new PersistenceManagerFactoryImpl();

        pmf.setConnectionURL(args[1]);
        pmf.setConnectionUserName(args[2]);
        pmf.setConnectionPassword(args[3]);

        try {
            InitialContext context = new InitialContext();
            context.bind(name, pmf);
        }

        catch (NamingException e) {
            System.out.println("Can't bind PersistenceManager");
            e.printStackTrace();
        }
    }
}

```

If not using JNDI then the singleton needs to be created and initialized directly. JDO-Helper can be used to do this based on a set of properties. The following code snippet taken from “PMFSingletonExample.java” shows how this could work. It uses an `initialize()` method that creates the `PersistenceManagerFactory` singleton from the specified properties:

```

import java.util.Properties;
import javax.jdo.*;

public class PMFSingletonExample {
    // PersistenceManagerFactory singleton

```

```

private static PersistenceManagerFactory pmf;

/*
 * This method returns the singleton
 * PersistenceManagerFactory.
 * If it the singleton hasn't been initialized it throws
a  * JDOFatalUserException.
 */
public static PersistenceManagerFactory
getPersistenceManagerFactory() {

    if (pmf == null) {

        throw new JDOUserException(
            "PersistenceManagerFactory not initialized.");
    }

    return pmf;
}

/*
 * Creates a PersistenceManagerFactory based on the
 * specified properties and Initializes the singleton.
 */
public static void initialize(
    String url, String name, String password) {

    Properties properties = new Properties();

    properties.put(
        "javax.jdo.PersistenceManagerFactoryClass", "XXX");
    properties.put(
        "javax.jdo.option.ConnectionURL", url);
    properties.put(
        "javax.jdo.option.ConnectionUserName", name);
    properties.put(
        "javax.jdo.option.ConnectionPassword", password);

    pmf =
        JDOHelper.getPersistenceManagerFactory(properties);
    }
}

```


Of course, before this example can be used the “XXX” strings would need to be replaced with values appropriate to the JDO implementation being used.

The previous example has the disadvantage that the `initialize()` method needs to be explicitly called with the appropriate connection properties. An alternative to this is shown in the following code snippet taken from “`PMFSingletonFromFileExample.java`”. It reads the properties from a file whose name is specified using a system property:

```
import java.util.Properties;
import java.io.*;
import javax.jdo.*;

public class PMFSingletonFromFileExample {

    // PersistenceManagerFactory singleton
    private static PersistenceManagerFactory pmf;

    /*
     * This method returns the singleton
     * PersistenceManagerFactory. If the singleton
     * hasn't been initialized it creates a
     * PersistenceManagerFactory from a properties
     * file denoted by the system property "jdo.properties"
     * and initializes the singleton.
     */
    public static PersistenceManagerFactory
    getPersistenceManagerFactory() {

        if (pmf == null) {

            String filename =
                System.getProperty("jdo.properties");

            if (filename == null) {
                throw new JDOFatalUserException(
                    "System property 'jdo.properties' not defined");
            }

            else {

                Properties properties = new Properties();

                try {

                    properties.load(new FileInputStream(filename));
```

```

        pmf =
            JDOHelper.
                getPersistenceManagerFactory(properties);
    }

    catch (java.io.IOException e) {
        throw new JDOFatalUserException(
            "Error reading '" + filename + "'", e);
    }
}
}

return pmf;
}
}

```

The following code snippet taken from “SupportedOptionsExample.java” shows how to print out the options that a `PersistenceManagerFactory` instance supports:

```

Iterator iter = pmf.supportedOptions().iterator();

System.out.println(
    "PersistenceManagerFactory class '" +
    pmf.getClass().getName() + "' supports:");

while (iter.hasNext()) {
    String option = (String) iter.next();

    System.out.println("  " + option);
}

```

Using the `getPersistenceManager()` methods is explained in the section on `PersistenceManager`.

5.3.2 PersistenceManager

`PersistenceManager` is the primary interface that a JDO application would use. It has a number of standard properties that govern how it manages persistent objects in memory and provides methods for an application to interact with those persistent objects and the underlying datastore. It also acts as a factory for `Query`, `Extent` and `Transaction` instances.

5.3.2.1 Creating a PersistenceManager

There are two main ways to create a PersistenceManager:

PersistenceManagerFactory The `getPersistenceManager()` methods on `PersistenceManagerFactory` return a `PersistenceManager` instance. For a managed application this is the only way to get a `PersistenceManager`. For a non-managed application this is the expected and portable approach to getting a `PersistenceManager`. A `PersistenceManager` got in this way will inherit its properties from its `PersistenceManagerFactory` and will remain associated with the `PersistenceManagerFactory` during its lifetime.

Constructors A JDO implementation may provide constructors that allow an application to directly create a `PersistenceManager`. This is suitable for non-managed applications, but has the disadvantage that it is non-portable and the code would need to be changed per JDO implementation.

5.3.2.2 PersistenceManager Properties

Table 5-5 lists the three properties defined by `PersistenceManager`:

Table 5-5 PersistenceManager Properties

Property	Method Summary
IgnoreCache	boolean <code>getIgnoreCache()</code> void <code>setIgnoreCache(boolean flag)</code> This property is a hint to the <code>PersistenceManager</code> as to whether it can ignore in-memory changes when executing a Query or iterating through an Extent.
Multithreaded	boolean <code>getMultithreaded()</code> void <code>setMultithreaded(boolean flag)</code> This property controls whether the <code>PersistenceManager</code> synchronizes internally to ensure that internal data structures do not get corrupted when used by multiple threads concurrently.
UserObject	Object <code>getUserObject()</code> void <code>setUserObject(Object obj)</code> This property allows the application to associate an arbitrary object with a <code>PersistenceManager</code> instance.

IgnoreCache Normally the result from a Query or Extent will reflect any in-memory changes made within the current transaction. If a new persistent object was created, or a persistent object was modified or deleted, then any result would reflect these changes. Generally this is the desired behavior. However it may come at a cost or be undesirable in

certain situations. Depending on the underlying datastore, it might be necessary to synchronize any in-memory changes with the datastore prior to executing the Query or iterating through the Extent.

As an optimization an application can choose not to incur this overhead, in which case the IgnoreCache property should be set to true. A result from a Query or iterating through an Extent may then not reflect in-memory changes made within the current transaction. If a new instance is created that matches a given query, it may not be part of the returned result. The advantage is that executing the query no longer requires in-memory changes to be synchronized with the datastore before hand.

This property is just a hint, even if set, it may be ignored by a JDO implementation. An application can't rely on a Query to ignore in-memory changes just because this property is true. Unless there is a specific performance requirement an application typically shouldn't set this property to true.

Multithreaded It is safe for an application to use multiple threads with JDO as long as there is only one thread using a given PersistenceManager or instances created by a PersistenceManager (Query, Extent, PersistenceCapable instances, and so on) at a given point in time. Simply put, if each thread has its own PersistenceManager there is no problem. However, if the application requires multiple threads to be able to use the same PersistenceManager instance at the same time then this property should be set to true.

If this property is true, the PersistenceManager will synchronize internally to ensure that internal data structures do not get corrupted when multiple threads use the PersistenceManager instance concurrently. Synchronization may incur additional overhead and adversely affect performance, this property should only be set if multi-threaded access is really required. See [chapter X](#) for more details on developing multithreaded applications.

5.3.2.3 Method Summary

PersistenceManager has methods that allow an application to interact with persistent objects and the underlying datastore and to get Query, Extent and Transaction instances. There are many methods, but the key ones that a JDO application is likely to use are:

Transaction currentTransaction()

The returned Transaction instance can be used to begin and end transactions.

void makePersistent(Object obj)

This makes a new instance of a persistence capable class persistent.

void deletePersistent(Object obj)

This deletes a persistent object in the datastore.

Query newQuery(Class cls, String filter)

The returned Query instance can be executed to find persistent objects based on the specified filter.

void close()

Indicates the PersistenceManager instance is no longer needed.

The PersistenceManager methods can be grouped into the following categories:

- Connection Management Methods
- Cache Management Methods
- Instance Management Methods
- Factory Methods

Connection Management Methods These methods allow the application to indicate that it no longer needs the PersistenceManager instance or get the associated PersistenceManagerFactory.

void close()

Indicates the PersistenceManager instance is no longer needed. Depending on whether the PersistenceManagerFactory implements pooling this may just return the PersistenceManager instance back to the PersistenceManagerFactory for reuse.

An attempt to use a PersistenceManager after it has closed will result in JDOFatalUserException being thrown.

Boolean isClosed()

Returns true if the PersistenceManager has been closed.

PersistenceManagerFactory getPersistenceManagerFactory()

Returns the PersistenceManagerFactory that created the PersistenceManager instance.

This may return null if the PersistenceManager was explicitly created by the application via a constructor.

Cache Management Methods Typically an application does not need to explicitly manage persistent objects in-memory; the JDO runtime implicitly takes care of them. In

some situations the application may need to explicitly release, retrieve or refresh a persistent object or group of persistent objects. A common reason for this is as a performance optimization – an application can explicitly retrieve a collection of persistent objects from the datastore into memory in one go, rather than retrieving them implicitly one by one.

The following methods all follow the same set of conventions. There is a base method that takes a single Object argument. Then there are additional methods whose names are post fixed with “All” to indicate they take multiple objects (either an Object[] or Collection argument). There may also be an “All” method that takes no argument, in which case it operates on all relevant in-memory instances.

If null used as an argument to a method that takes Object then the null is ignored and the method is effectively a no-op. If null is passed to a method that takes an Object[] or Collection then NullPointerException will be thrown. If the Object[] or Collection argument itself contains null elements then these are ignored.

void evict(Object obj)

void evictAll(Object[] objs)

void evictAll(Collection objs)

void evictAll()

By default persistence objects are automatically evicted by the PersistenceManager at the end of a transaction. However, the evict methods allow an application to provide a hint to the PersistenceManager that the specified instances are no longer required in-memory and they can be made available for garbage collection during a transaction (although they will only be garbage collected if actually no longer referenced by the application).

These methods will evict instances even if the RetainValues property is true for the transaction (see [section X.X](#) for details on RetainValues) and evicted instances transition to being hollow. The evictAll() method with no argument will evict all unmodified in-memory (persistent -clean) instances.

This is useful for applications that need to retrieve many persistent objects, more than would normally fit into memory, during a transaction. The application can call the evict methods to release instances once it has finished with them so they can potentially be garbage collected during the transaction and make room for other persistent objects.

Typically it only makes sense to evict unmodified instances, although evicting a modified instance is guaranteed to not cause any loss of in-memory changes. An

implementation may choose to ensure this by ignoring the eviction of modified instances. Some implementations may support eviction of modified instances, perhaps by storing the changes in the underlying datastore.

These methods are generally useful within a transaction, however if the `RetainValues` property is true then these methods can be used to evict specific instances after a transaction ends also.

Garbage Collection

During a transaction a `PersistenceManager` needs to maintain a reference to all in-memory persistent objects, even if the application no longer references them the JDO runtime will (although some implementations may choose to use weak references). This means that during a transaction in-memory persistent objects typically won't be available for garbage collection.

Eviction allows the garbage collector to do its job and collect instances that are no longer referenced by the application.

`void refresh(Object obj)`

`void refreshAll(Object[] objs)`

`void refreshAll(Collection objs)`

`void refreshAll()`

The refresh methods re-retrieve the values of the fields from the datastore for the specified instances.

These methods will re-retrieve field values even for instances that have been modified (persistent-dirty), in which case any in-memory changes are lost. Refreshed instances transition to being in-memory and unmodified (persistent-clean).

Typically these methods only make sense for optimistic transactions (see [section X.X](#) for more details on optimistic transactions) where the in-memory state of a persistent object is not necessarily up to date with the datastore. These methods allow an application to refresh an instance, perhaps before making a modification, to reduce the likelihood of a conflicting update being detected when the transaction ends.

These methods can typically only be used within a transaction, unless a JDO implementation supports non-transactional read, in which case they can be used out-

side of a transaction also. If used outside of a transaction, `refreshAll()` is a no-op.

`void retrieve(Object obj)`

`void retrieve(Object obj, boolean flag)*`

`void retrieveAll(Object[] objs)`

`void retrieveAll(Object[] objs, boolean flag)*`

`void retrieveAll(Collection objs)`

`void retrieveAll(Collection objs, boolean flag)*`

The retrieve methods retrieve the values of all the fields from the datastore for the specified instances. By default these methods will retrieve values for all fields, not just those in the default fetch group. A flag can optionally be used to control this behavior, if true all field values are retrieved, if false only values for fields in the default fetch group are retrieved.

Unlike the refresh methods retrieve will not overwrite field values that have already been retrieved, therefore modified fields will not be overwritten.

These methods have two primary uses. One is to ensure that all the field values for an instance are in-memory. This is useful before trying to make a persistent object transient. If a persistent object is made transient before retrieving all of its field values then some the fields will have their default values only.

The second is as a performance optimization. This is useful when iterating through the result of a query, rather than retrieving each object individually an application can call a retrieve method and retrieve them all at once, avoiding multiple calls to the underlying datastore (one per instance).

These methods can typically only be used within a transaction, unless a JDO implementation supports non-transactional read, in which case they can be used outside of a transaction also.

*These methods are defined in the 1.0.1 maintenance revision to the JDO specification and may not be supported by all JDO implementations.

Instance Management Methods These methods allow an application to interact with the `PersistenceManager` and `PersistenceCapable` instances that it manages:

Object `getObjectId(Object obj)`

Object `getTransactionalObjectId(Object obj)`

Every persistent object has an identity associated with it. An application can get the identity of a persistent object by calling the `getObjectId()` method.

If a persistence capable class uses application identity then it may be possible to modify a persistent object's identity during a transaction (by modifying the values of the primary key fields). The `PersistenceManagerFactory` option, `javax.jdo.option.ChangeApplicationIdentity`, denotes if this is supported or not. If a persistent object's identity is changed during a transaction then `getObjectId()` will return the original identity, as it was at the start of the transaction. If an application needs the changed identity then it can call `getTransactionalObjectId()`. If changing application identity is not supported or the application identity hasn't been changed or application identity isn't being used then calling `getTransactionalObjectId()` is the same as calling `getObjectId()`.

These methods are useful to get the identity of a persistent object and use this to retrieve the persistent object at a later point in time or using a different `PersistenceManager`, by calling `getObjectById()`.

These methods can be used both in and outside of a transaction. If used outside of a transaction, `getTransactionalObjectId()` is equivalent to `getObjectId()`.

Object Identity

The object identity returned from `getObjectId()` is essentially an opaque type, the application need know nothing about it. If using application identity the implementer of the persistence capable class is responsible for providing the object identity class, otherwise the class is provided by the JDO implementation itself. Regardless, any object identity class has a number of characteristics the application can rely on.

One of these is that it must implement `Serializable`; this allows a persistent object's identity to be sent to another application or stored somewhere for later use.

Another one is that the `equals()` and `hashCode()` methods for the class have to work based on the underlying object identity being represented. This allows an application to compare two persistence objects from different `PersistenceManager` instances based on their identities.

Yet another is that it must have a constructor that takes a single string argument that recreates the object identity based on what was returned by the identity class' toString() method. The constructor is used by the newObjectIdInstance() method on PersistenceManager to recreate an object identity from a string. This allows an application to convert an object identity to a string that can be later used to recreate it, avoiding needing to serialize the object identity instance itself.

Class getObjectIdClass(Class pcClass)

If a persistence capable class uses application identity then this method can be used to retrieve the actual class used to represent the object identity for the class. If the specified class does not use application identity or is null then this method returns null.

This method is only useful if the application needs to know for some reason the class that implements the object identity for a persistence capable class.

This method can be used both in and outside of a transaction.

Object newObjectIdInstance(Class pcClass, String oid)

Object getObjectById(Object oid, boolean validate)

These methods can be used to return a persistent object given its identity. If the persistence capable class uses application identity then these methods can be used between PersistenceManager instances from different JDO implementations, otherwise they only work across PersistenceManager instances from the same JDO implementation.

The newObjectIdInstance() method is used to create an object identity instance from a string previously returned by the toString() method. The persistence capable class of the persistent object whose identity is represented by the specified string needs to be provided.

The getObjectById() method returns the persistent object represented by the specified object identity instance. If the persistent object is already in-memory a reference to it is simply returned. If it is not, an in-memory instance is created with the specified identity. Whether the persistent object is actually retrieved or not from the

datastore is dependent on the JDO implementation and the validate flag.

If validate is true then the PersistenceManager checks that a persistent object with the specified identity actually exists, and if called within a transaction actually retrieves it from the datastore. If there is no persistent object with the specified identity in the datastore then JDODatastoreException is thrown.

If validate is false it is up to the JDO implementation as to whether it validates whether there is actually a persistent object in the datastore with the specified identity and whether it retrieves it or not. If this method doesn't validate the persistent object exists, then JDODatastoreException will be thrown when the in-memory instance is used instead.

These methods can be used both in and outside of a transaction.

void makePersistent(Object obj)

void makePersistentAll(Object[] objs)

void makePersistentAll (Collection objs)

These methods can be used to explicitly make transient instances persistent so they will be stored in the datastore after the transaction successfully completes. A transient instance transitions to being a new persistent object (persistent-new) as a result of this method. If a specified instance is already persistent it is left unchanged, although if it belongs to another PersistenceManager instance JDOUserException is thrown.

After the transaction is committed all persistent objects transition to being hollow. If the transaction rolls back instead, then any new persistent objects (persistent-new) will transition back to being transient again.

These methods can only be used within a transaction, if called outside of a transaction JDOUserException is thrown.

void deletePersistent(Object obj)

void deletePersistentAll(Object[] objs)

void deletePersistentAll (Collection objs)

These methods can be used to explicitly delete persistent objects such that they will no longer be stored in the datastore after the transaction successfully commits. A persistent object transitions to being a deleted persistent object (persistent-deleted or persistent-new-deleted) as a result of this method. If a specified instance has already been deleted within the transaction it is ignored. If an instance belongs to a different PersistenceManager instance or is transient JDOUserException is thrown.

After the transaction is committed, deleted persistent objects transition to being transient and lose their object identity. If the transaction rolls back instead then deleted persistent objects transition to being a hollow again.

These methods can only be used within a transaction, if called outside of a transaction JDOUserException is thrown.

Cascade Delete

Unlike making objects persistent, deleting objects only deletes the specified instances. There is no reachability algorithm, referenced persistent objects are not deleted (although some JDO implementations may provide vendor extensions that do this).

If an application needs to ensure referenced persistent objects are also deleted then a persistence capable class can implement the `jdoPreDelete()` method defined by the InstanceCallbacks interface and explicitly delete the referenced instances. See [section X.X](#) on InstanceCallbacks for more details.

void makeTransient(Object obj)**void makeTransientAll(Object[] objs)****void makeTransientAll (Collection objs)**

These methods can be used to detach a persistent object from its PersistenceManager and make it transient. This allows the instance to be used outside of the context of PersistenceManager instance. These methods do not affect persistent object in the underlying datastore, it's just a change to the in-memory instance itself. The instance retains any field values as-is but no longer has any object identity.

The effect of this method is immediate and is not subject to rollback. If a modified persistent (persistent-dirty) object is made transient `JDOUserException` is thrown. If an instance is already transient it is ignored.

These methods are useful if an application needs to move a persistent object to a different `PersistenceManager` or pass it to non-JDO application. If required later, the application will need to maintain the object identity of the persistent object, since when it is made transient it loses its identity. Because the field values of the persistent object are left as-is when it is made transient, the application should ensure all the field values have been retrieved from the datastore prior to making it transient. This can be done using retrieve methods on `PersistenceManager`.

`void makeTransactional(Object obj)`

`void makeTransactionalAll(Object[] objs)`

`void makeTransactionalAll(Collection objs)`

These methods can be used to make a transient or persistent but non-transactional instance transactional.

If the specified instances are transient and the JDO implementation supports the transient transactional optional feature then the instances transition to being transient transactional (transient-clean). If not supported `JDOUnsupportedOptionException` is thrown. When used with transient instances these methods can be used both in and outside of a transaction.

If the specified instances are non-transactional persistent objects then they transition to being persistent again (persistent-clean). The field values of the persistent objects are re-retrieved from the datastore. When used with non-transactional persistent objects these methods can only be called within a transaction, otherwise `JDOUserException` is thrown.

The affect of these methods is immediate and not subject to rollback (i.e. once made transient transactional an instance remains so, even if the transaction is rolled back).

void makeNontransactional(Object obj)

void makeNontransactionalAll(Object[] objs)

void makeNontransactionalAll (Collection objs)

These methods can be used to make transient transactional or persistent instances non-transactional.

If the specified instances are transient transactional then they transition to being transient. If an instance has been modified (transient-dirty) within the current transaction then `JDOUserException` is thrown.

If the specified instances are persistent and the JDO implementation supports the non-transactional optional feature then they transition to being persistent, non-transactional (persistent-nontransactional). If not supported `JDOUnsupportedOperationException` is thrown. If an instance has been modified (persistent-dirty) within the current transaction then `JDOUserException` is thrown.

The affect of these methods is immediate and not subject to rollback (i.e. once made non-transactional an instance remains so, even if the transaction is rolled back).

These methods can be used both in and outside of a transaction.

Factory Methods PersistenceMananger acts as a factory for Query, Extent and Transaction instances. The following factory methods can be used to construct these instances:

Query newQuery()

Query newQuery(Object query)

Query newQuery(String language, Object query)

Query newQuery(Class cls)

Query newQuery(Extent extent)

Query newQuery(Class cls, Collection cln)

Query newQuery(Class cls, String filter)

Query newQuery(Class cls, Collection cln, String filter)

Query newQuery(Extent extent, String filter)

These method returns a Query instance that can be used to find persistent objects based on a filter. See [section X.X](#) for more details.

These methods can be used both in and outside of a transaction.

Extent `getExtent(Class cls, boolean subclasses)`

This method returns an Extent instance that represents all the persistent objects of the specified class in the datastore. See [section X.X](#) for more details.

If subclasses is true then the returned Extent also represents all persistent objects of the specified class and all subclasses. If false the Extent only represents persistent objects of the specified class itself.

This method can be used both in and outside of a transaction.

Transaction `currentTransaction()`

This method returns the current Transaction instance associated with the PersistenceManager. This allows the application to begin and end transactions. See [section X.X](#) for more details.

5.3.2.4 Usage Guidelines

The following examples demonstrate how to use the various APIs defined on PersistenceManager:

Connection Management Methods Getting a PersistenceManager instance is straightforward, however for a server-side application care needs to be taken to ensure the PersistenceManager instance is always closed. The easiest way to do this is to wrap the use of a PersistenceManager instance within a try block and use a finally block to ensure the PersistenceManager instance gets closed even if an exception is thrown. The finally block can also ensure that any active transaction is first rolled back before the PersistenceManager instance is closed.

The following code snippet taken from “PersistenceManagerExample.java” shows how to use try and finally to ensure a PersistenceManager instance is closed:

```
PersistenceManager pm = null;  
  
try {
```

```

    pm = pmf.getPersistenceManager();
    // Do something interesting
    pm.close();
}
finally {
    if (pm != null && !pm.isClosed()) {
        if (pm.currentTransaction().isActive()) {
            pm.currentTransaction().rollback();
        }
        pm.close();
    }
}

```

Transactions The `currentTransaction()` method returns the `Transaction` instance for the `PersistenceManager` instance. For a non-managed application the transaction instance can be used to begin and end transactions. The following code snippet taken from “`TransactionExample.java`” shows how to begin and commit a transaction:

```

Transaction tx = pm.currentTransaction();

tx.begin();

// Do something interesting

tx.commit();

```

Extents The `getExtent()` method can be used to get a collection of all the instances of a given persistence capable class (and optionally subclasses) in the datastore. The following code snippet taken from “`ExtentExample.java`” shows how to iterate through all instances of the `Book` class (and subclasses):

```

Extent extent = pm.getExtent(Book.class, true);

Iterator books = extent.iterator();

System.out.println("Listing of all books:");

```



```
while (books.hasNext()) {  
    Book book = (Book) books.next();  
    System.out.println(" " + book.getTitle());  
}  
extent.close(books);
```

Evict Methods The previous example retrieved all instances of the `Book` class, however if there are a large number of books in the datastore then they may not all fit into memory at the same time. The evict methods can be useful when an application needs to access a large number of persistent objects within a single transaction but not all instances can possibly fit in memory. The following code snippet taken from “`EvictExample.java`” shows how to use the `evict()` method to evict each `Book` instance once it has been accessed:

```
Extent extent = pm.getExtent(Book.class, true);  
Iterator books = extent.iterator();  
System.out.println("Listing of all books:");  
while (books.hasNext()) {  
    Book book = (Book) books.next();  
    System.out.println(" " + book.getTitle());  
    pm.evict(book);  
}  
extent.close(books);
```

Retrieve Methods The previous example retrieved each persistent object one at a time from the datastore. As an optimization the `retrieveAll()` method can be used to potentially retrieve multiple persistent objects from the datastore in one go. This can improve the performance of the application by reducing the number of times the underlying datastore needs to be accessed. The following code snippet taken from “`RetrieveExample.java`” shows how the `retrieveAll()` method can be used to retrieve a group of instances:

```
Extent extent = pm.getExtent(Book.class, true);  
Iterator books = extent.iterator();
```

```

System.out.println("Listing of all books:");
List list = new ArrayList(1024);
while (books.hasNext()) {
    list.add(iter.next());
    if (list.size() == 1024 || !books.hasNext()) {
        pm.retrieveAll(list);
        for (int i = 0; i < list.size(); ++i) {
            Book book = (Book) list.get(i);
            System.out.println("  " + book.getTitle());
        }
        pm.evictAll(list);
        list.clear();
    }
}
extent.close(books);

```

If supported a further optimization would be to use a `retrieveAll()` method that takes a boolean flag to indicate that only the fields in the default fetch group are required. These methods were introduced in the 1.0.1 maintenance revision and so may not be supported by all JDO implementations initially.

Timing the two examples “`EvictExample.java`” and “`RetrieveExample.java`” using the same dataset shows how retrieving objects can dramatically improve the performance of an application. A simple test using 100,000 books shows that using `retrieveAll()` is **X.X** times faster than retrieving persistent objects one by one.

Refresh Methods These methods are primarily useful when using an optimistic transaction or using non-transactional instances to re-retrieve the fields of an instance from the datastore. However, another use is as a way to undo an in-memory modification to a persistent object within a transaction without rolling back the entire transaction. If an application decides that a change should not have been made to a particular instance it is possible to use `refresh()` to re-retrieve the instance’s fields from the datastore and thus overwrite any modified fields. The following code snippet taken from “`RefreshExample.java`” shows how `retrieve()` can be used to undo a change made to a book’s title:

```
tx.begin();

Book book =
    new Book(
        "Core Java Data Objects",
        "0-13-140731-7",
        new GregorianCalendar(
            2003, Calendar.JUNE, 1).getTime());

tx.commit();

System.out.println(
    "Book's title is '" + book.getTitle() + "'.");

book.setTitle("Core JDO");

System.out.println(
    "Book's title changed to '" + book.getTitle() + "'.");

pm.refresh(book);

System.out.println(
    "Book's title after refresh is '" +
    book.getTitle() + "'.");

tx.commit();
```

The output would be:

```
Book's title is 'Core Java Data Objects'.
Book's title changed to 'Core JDO'.
Book's title after refresh is 'Core Java Data Objects'.
```

Object Identity Methods A persistence object's identity is represented by an instance of a normal Java class. An object identity instance is a useful way to pass a reference to a persistent object between `PersistenceManager` instances and even between `PersistenceManager` instances in different JVMs (since it is serializable).

The following code snippet taken from "ObjectIdentityExample.java" shows how to get the identity of a persistent object and use it to create a reference to the persistent object in a different `PersistenceManager` instance:

```
PersistenceManager pm = pmf.getPersistenceManager();

Transaction tx = pm.currentTransaction();
```

```
tx.begin();

Author author =
    new Author(
        "Keiron McCammon",
        new Address(
            "6539 Dumbarton Circle", "Fremont", "CA", "94555"));

pm.makePersistent(author);

tx.commit();

Object oid = pm.getObjectId(author);

System.out.println("Author's object identity is: " + oid);

pm.close();

PersistenceManager pm2 = pmf.getPersistenceManager();

Transaction tx2 = pm2.currentTransaction();

tx2.begin();

Author author2 = (Author) pm.getObjectById(oid, true);

System.out.println("Author is: " + author2.getName());

tx2.commit();

pm2.close();
```

Although the above example shows how to use a persistent object's identity to re-create a reference to it in a different `PersistenceManager` instance within the same JVM it could easily have been passed via RMI to a different JVM altogether. In this case the object identity instance just gets serialized and passed over the wire.

In some situations it is desirable to pass the identity of a persistent object to a non-JDO application, this application would then use the identity as a parameter in additional requests (a web browser for example). In this situation the object identity instance can be converted to a string representation. The following code snippet taken from `"StringObjectIdentityExample.java"` shows how to convert a persistent object's identity to a string and re-create a reference to the persistent object again from the string:

```
Transaction tx = pm.currentTransaction();
```

```
tx.begin();

Author author =
    new Author(
        "Keiron McCammon",
        new Address(
            "6539 Dumbarton Circle", "Fremont", "CA", "94555"));

pm.makePersistent(author);

tx.commit();

String oid = pm.getObjectId(author).toString();

System.out.println("Author's object identity is: " + oid);

pm.close();

PersistenceManager pm2 = pmf.getPersistenceManager();

Transaction tx2 = pm2.currentTransaction();

tx2.begin();

Author author2 =
    (Author) pm.getObjectById(
        pm.newObjectIdInstance(Author.class, oid),
        true);

System.out.println("Author is: " + author2.getName());

tx2.commit();
```

The `toString()` method is used to convert the persistent object's identity to a string. In a different `PersistenceManager` instance the `newObjectIdInstance()` is used to re-create an object identity instance from the string and this is then used to get a reference to the original persistent object.

Make and Delete Persistent Methods The `makePersistent()` and `deletePersistent()` methods create and delete the persistent objects in the underlying datastore.

The `makePersistent()` methods mark in-memory transient instances of persistence capable classes as persistent. The methods can be called at any point within a transaction and because of persistence by reachability only the root of a graph of interconnected instances needs to be made persistent. The following code snippet taken from "MakePersistentExam-

ple.java” creates instances of Author, Address, Book and Publisher and associates them together. Because an Author references an Address and a Book and a Book references a Publisher only the Author instance needs to be made persistent. However, it wouldn’t matter if the application made the Address, Book and Publisher instances explicitly persistent also (which might be the case if there is doubt about what is reachable from where):

```
tx.begin();

Author author =
    new Author(
        "Keiron McCammon",
        new Address(
            "6539 Dumbarton Circle", "Fremont", "CA", "94555"));

Publisher publisher =
    new Publisher("Sun Microsystems Press");

Book book =
    new Book(
        "Core Java Data Objects",
        "0-13-140731-7",
        new GregorianCalendar(
            2003, Calendar.JUNE, 1).getTime());

author.addBook(book);

publisher.addBook(book);

pm.makePersistent(author);

tx.commit();
```

The deletePersistent() methods mark in-memory persistent objects as deleted. The methods can be called at any point during a transaction but unlike makePersistent() they only affect the specific persistent object being deleted. It is the responsibility of the application to delete any referenced persistent objects that are no longer required. The following code snippet taken from “DeletePersistentExample.java” gets the first Author instance from the class Extent, deletes the associated Address instance and removes the Author from each of its books:

```
Iterator authors = extent.iterator();

if (authors.hasNext()) {

    Author author = (Author) authors.next();
```

```
pm.deletePersistent(author.getAddress());

Iterator books = author.getAllBooks();

while (books.hasNext()) {
    Book book = (Book) books.next();
    book.removeAuthor(author);
}

pm.deletePersistent(author);
}
```

Rather than relying on the application to remember to tidy up every time it deletes a persistent object a persistence capable class can implement the `InstanceCallbacks` interface and use the `jdoPreDelete()` method to tidy up automatically upon deletion. See [section X.X](#) for more details.

Make Transient Methods An in-memory persistent object can be made transient using the `makeTransient()` methods. Once made transient the in-memory instance has no object identity and is no longer associated with a `PersistenceManager`. Because the fields of the instance are left as-is the application should first use one of the `retrieve()` methods to ensure all the fields of the instance have been retrieved from the datastore. The following code snippet taken from “`MakeTransientExample.java`” gets the first `Author` instance from the class `Extent`, retrieves all of its fields and makes it transient. Even after the `PersistenceManager` instance is closed the transient `Author` instance can still be used:

```
PersistenceManager pm = pmf.getPersistenceManager();

Transaction tx = pm.currentTransaction();

tx.begin();

Extent extent = pm.getExtent(Author.class, true);

Iterator authors = extent.iterator();

Author author = null;

if (authors.hasNext()) {
    author = (Author) authors.next();
}
```

```

    pm.retrieve(author);
    pm.makeTransient(author);
}

extent.close(authors);

tx.commit();

pm.close();

if (author != null) {
    System.out.println(
        "Author's name is: " + author.getName());
}

```

The `makeTransient()` methods only affect the specific instance being made transient. If referenced persistent objects should also be made transient then it is the application's responsibility to make each instance explicitly transient.

Usage Warning

Care must be taken when making instances transient. If an in-memory persistent object that has been modified references an instance that was made transient, then at commit, due to persistence by reachability, the transient instance will be treated as a new persistent object.

It is the applications responsibility to ensure that there are no references to the transient instance. For this reason great care must be taken when making persistent objects transient.

Transactional and Non-transactional Methods The behavior of these methods depends on whether a transient or persistent instance is used.

The following code snippet taken from “`TransientTransactionalExample.java`” shows how a transient instance of a persistence capable class can be made transactional so that on rollback the instance's fields revert to their values at the beginning of the transaction and how when made non-transactional again the fields are left as-is on rollback:

```

Address address =
    new Address(
        "6539 Dumbarton Circle", "Fremont", "CA", "94555");

System.out.println("Making instance transactional.");

pm.makeTransactional(address);

```



```
Transaction tx = pm.currentTransaction();

tx.begin();

System.out.println(
    "Zipcode is '" + address.getZipcode() + "'");

address.setZipcode("99999");

System.out.println(
    "Modified zipcode is '" + address.getZipcode() + "'");

tx.rollback();

System.out.println(
    "Zipcode after rollback is '" +
    address.getZipcode() + "'");

System.out.println("Making instance non-transactional.");

pm.makeNontransactional(address);

tx.begin();

System.out.println(
    "Zipcode is '" + address.getZipcode() + "'");

address.setZipcode("99999");

System.out.println(
    "Modified zipcode is '" + address.getZipcode() + "'");

tx.rollback();

System.out.println(
    "Zipcode after rollback is '" +
    address.getZipcode() + "'");
```

The output would be:

```
Making instance transactional.
Zipcode is '94526'.
Modified zipcode is '99999'.
Zipcode after rollback is '94526'.
Making instance non-transactional.
```

```
Zipcode is '94526'.  
Modified zipcode is '99999'.  
Zipcode after rollback is '99999'.
```

The following code snippet taken from “PersistentTransactionalExample.java” shows how a persistent object can be made non-transactional and accessed outside of the transaction and even modified and how when the instance is made transactional again any modified fields are simply discarded:

```
tx.begin();  
  
Address address =  
    new Address(  
        "6539 Dumbarton Circle", "Fremont", "CA", "94555");  
  
pm.makePersistent(address);  
  
tx.commit();  
  
System.out.println("Making instance non-transactional.");  
  
pm.makeNontransactional(address);  
  
System.out.println(  
    "Zipcode is '" + address.getZipcode() + "'");  
  
address.setZipcode("99999");  
  
System.out.println(  
    "Modified zipcode is '" + address.getZipcode() + "'");  
  
tx.begin();  
  
System.out.println("Making instance transactional.");  
  
pm.makeTransactional(address);  
  
System.out.println(  
    "Transactional zipcode is '" +  
        address.getZipcode() + "'");  
  
tx.commit();
```

The output would be:

```
Making instance non-transactional.  
Zipcode is '94526'.  
Modified zipcode is '99999'.  
Making instance transactional.  
Transactional zipcode is '94526'.
```

5.3.3 Extent

An Extent instance essentially represents a collection of all the persistent objects of a persistence capable class in the datastore. An Extent can be used to iterate through all the persistent objects of the class in no particular order or used to construct a Query.

By default the JDO runtime in conjunction with the underlying datastore maintains an Extent for every persistence capable class. It is possible to provide a hint in the JDO metadata that an Extent is not required by explicitly specifying the “requires-extent” attribute as “false”. Depending on the JDO implementation this may or may improve performance.

Some datastores don’t implicitly maintain a collection of all persistent objects being stored per class, in which case a collection of instances must be explicitly maintained instead. This can add additional overhead. If the application has no need to get an Extent or Query the instances of a class then it can avoid any unnecessary overhead by setting the “requires-extent” attribute to false in the metadata for the class.

For many datastores Extent management is implicit (for example a relational database always knows all the rows in a table). In this case setting the “requires-extent” attribute to false makes no difference.

The Extent instance itself doesn’t actually reference persistent objects, rather it just maintains a reference to a persistence capable class, a subclasses flag and a collection of open Iterators. It’s the Iterators created by the Extent or Query instances created using the Extent that return the persistent objects.

Since the instances of persistence capable classes can vary overtime as different applications create and delete persistent objects Iterators created by an Extent at different points in time may return different sets of persistent objects. An Iterator will return the persistent objects in the datastore at the time the Iterator was created. Depending on the IgnoreCache property of the PersistenceManager the Iterator may or may include instances made persistent or deleted within the current transaction.

5.3.3.1 Creating an Extent

An Extent is created using the `getExtent()` method on `PersistenceManager`:

```
public Extent getExtent(Class cls, boolean subclasses)
```

The boolean flag controls whether the Extent contains persistent objects of just the specified class or includes persistent objects of all subclasses also.

5.3.3.2 Extent Properties

Extent defines three read-only properties that are set when the Extent is created:

Table 5-6 Extent Properties

Property	Method Summary
CandidateClass	Class <code>getCandidateClass()</code> This property denotes the persistence capable class specified when the Extent instances was created.
PersistenceManager	PersistenceManager <code>getPersistenceManager()</code> This property denotes the PersistenceManager that created the Extent instance.
Subclasses	boolean <code>hasSubclasses()</code> This property determines if the Extent contains persistent objects of the specified class and its subclasses or not. If true then the Extent includes all persistent objects of the specified class and all subclasses. If false it includes persistent objects of the specified class only.

5.3.3.3 Method Summary

Extent provides a method to get an Iterator (that can then be used to iterate through all the persistent objects represented by the Extent) and methods to close Iterators after they have been used.

void close(Iterator iter)

This method closes the specified Iterator. Once closed the Iterator's `hasNext()` method will return false and `next()` will throw `NoSuchElementException`.

void closeAll()

This method closes all Iterators that have been created from the Extent. Once closed any Iterator's `hasNext()` method will return false and `next()` will throw `NoSuchElementException`.

This method only closes open Iterators, the Extent itself is still valid and can be used to create new Iterators or Query instances.

Iterator iterator()

This method returns an Iterator. The Iterator can be used to iterate through all the persistent objects represented by the Extent.

If the IgnoreCache property is false then the Iterator will return instances that have been made persistent within the current transaction and won't return instances that have been deleted within the current transaction. As an optimization, if the PersistenceManager instances IgnoreCache property is true then the returned Iterator may or may not return instances that have been made persistent within the current transaction and may or may not return instances that have been deleted within the current transaction.

When an application has finished with an Iterator it be closed using the close() method. Alternatively all Iterators can be closed in one go using the closeAll() method.

The returned Iterator does not implement the remove() method, if called UnsupportedOperationException will be thrown.

Closing Iterators

It is best practice to ensure that any Iterator is closed when finished with. Depending on the JDO implementation this may release datastore resources used by the Iterator (a database cursor for example).

5.3.3.4 Usage Guidelines

The following code snippet taken from "ExtentExample.java" shows how to iterate through all instances of the Book class, closing the Iterator when finished:

```
Extent extent = pm.getExtent(Book.class, true);

Iterator books = extent.iterator();

System.out.println("Listing of all books:");

while (books.hasNext()) {

    Book book = (Book) books.next();

    System.out.println("  " + book.getTitle());

}
```

```
extent.close(books);
```

5.3.4 Query

JDO defines both a query API and a query language. The API allows an application to construct and execute a query programmatically and the query language allows an application to specify the persistent objects it wants. The query language is called JDO-QL (JDO Query Language) and its syntax is Java boolean expression.

Essentially a JDO application uses a JDO-QL string (referred to as a filter) to create a Query instance, this instance is then executed and returns a collection of persistent objects that satisfy the specified filter.

As well as its own query language JDO allows implementations to offer additional query languages. An application would use the same Query APIs defined by JDO, however the syntax of the query string would instead be SQL, OQL or some other proprietary language. JDO mandates support for JDO-QL but leaves support for additional languages as optional. An implementation that supports additional query languages would include them in the list of supported options returned from the `supportedOptions()` method on `PersistenceManager`.

A Query consists of a set of candidate instances, which can be specified using a Java class an `Extent` or a Java collection, and a filter string. In addition it is possible to also declare import statements, parameters and variables as well as an ordering for the set of results. When executed a Query takes the set of candidate instances and returns a Java collection of references to the instances that satisfy the query filter.

The goal of JDO-QL is to provide a simple query grammar that is familiar to Java programmers and that can be executed by the JDO implementation, possibly by converting it to a different representation and passing it to the underlying datastore. JDO makes no assumptions about where the query is actually executed. Depending on the JDO implementation and the underlying datastore being used, a query may be executed by retrieving all the persistent objects into memory or it may simply be passed to the underlying datastore.

See [chapter X](#) for more details on the JDO-QL syntax.

5.3.4.1 Creating a Query

A Query is created using the `newQuery()` methods on `PersistenceManager`. There are essentially five ways to create a Query:

- Create an empty one and specify the candidate instances and filter later.
- Create a copy of another one.
- Create one where the candidate instances are denoted by a Java class.
- Create one where the candidate instances are denoted by an `Extent`.

- Create one where the candidate instances are denoted by a Java collection.

The set of candidate instances for a Query should be specified one way, using a Java class, an Extent or a Java collection.

Query newQuery()

Creates an empty Query.

The candidate instances and filter have to be specified before the query can be executed.

Query newQuery(Object query)

Creates a Query as a copy of the specified Query.

This is useful when creating a Query based on an existing Query owned by a different PersistenceManager. It can also be used to create a Query based on a Query that was previously de-serialized.

Query newQuery(String language, Object query)

Creates a Query for the specified query language.

The string used to denote the language being used and the query parameter are both implementation specific.

Query newQuery(Class cls)**Query newQuery(Class cls, String filter)**

Creates a Query using the specified Class to denote the set of candidate instances.

The candidate instances for a Query created using a class include all persistent objects in the datastore of the specified persistence capable class and all subclasses.

Query newQuery(Extent extent)**Query newQuery(Extent extent, String filter)**

Creates a Query using the specified Extent to denote the set of candidate instances.

If the subclasses flag for the Extent is false then the set of candidate instances includes persistent objects in the datastore of persistence capable class used to create the

Extent only. Otherwise the candidate instances include persistent objects of all subclasses also.

Query newQuery(Class cls, Collection cln)

Query newQuery(Class cls, Collection cln, String filter)

Creates a Query using the specified Collection to denote the set of candidate instances. The specified Class denotes the persistence capable class of the instances contained in the collection.

A Query created using a Collection will query on only the instances contained in the collection.

5.3.4.2 Query Properties

Query defines two properties:

Table 5-6 Query Properties

Property	Method Summary
IgnoreCache	boolean getIgnoreCache() void setIgnoreCache(boolean flag) This property is a hint as to whether the Query can ignore in-memory changes when executed. The initial value is the same as the setting of the PersistenceManager instance's IgnoreCache property at the time the Query was constructed.
PersistenceManager	PersistenceManager getPersistenceManager() This property denotes the PersistenceManager used to create the Query.

5.3.4.3 Method Summary

The Query methods can be grouped into the following categories:

- Query Definition Methods
- Query Execution Methods

Query Definition Methods The following methods are used to define a Query. Before a Query can be executed a set of candidate instances and a filter have to be set as a minimum.

void declareImports(String imports)

It is possible to provide import statements that are used to resolve type ambiguities within a filter expression.

The imports string is a list of Java import statements separated by semicolons (as per normal Java import statements).

void declareParameters(String params)

It is possible to declare parameters that can be used within a filter expression. The parameters are substituted for actual values when the Query is executed.

The params string is a list of Java parameters (type and identifier) separated by commas (as per normal Java parameter declarations for a method).

void declareVariables(String vars)

It is possible to declare variables that can be used within a filter expression.

The vars string is a list of Java local variables separated by semicolons (as per normal Java local variable declarations).

void setClass(Class cls)

This method sets the candidate class for the Query as the specified persistence capable class.

If a setCandidates() method is not used then the set of candidate instances will be all those persistent objects in the datastore belonging to the specified class and its subclasses.

void setCandidates(Extent extent)

This method specifies the set of candidate instances as those belonging to the specified Extent.

If the subclasses flag for the Extent is false then the set of candidate instances includes persistent objects in the datastore of the persistence capable class used to create the Extent only. Otherwise the candidate instances include persistent objects of all subclasses also.

There is no need to call `setClass()` since an `Extent` implicitly specifies the candidate class for the `Query` as the persistence capable class used to create the `Extent`.

`void setCandidates(Collection collection)`

This method restricts the set of candidate instances for a class to those contained in the collection.

The `setClass()` method must be used to specify the persistence capable class of the instances in the collection.

`void setFilter(String filter)`

This method sets the filter for the `Query`.

See chapter X, JDO-QL for more details on the format of the filter string.

`void setOrdering(String ordering)`

It is possible to specify that persistent objects should be returned sorted in an ascending or descending order.

The ordering string is a list of fields, followed by the “ascending” or “descending” keyword, each separated by a comma.

Query Execution Methods The following methods are used to execute a `Query` and get a collection of persistent objects that satisfy the filter.

`void close(Object result)`

This method closes the result returned from an `execute()` method. Any Iterators retrieved from a result after it has been closed will return false to `hasNext()` and `next()` will throw `NoSuchElementException`.

`void closeAll()`

This method closes all the results returned from an `execute()` method.

The `Query` itself can still be used, only the results returned by the `execute()` methods are closed.

`void compile()`

This method will validate that the Query is valid, if not `JDOUserException` will be thrown.

It also acts as a hint to the JDO implementation that the Query should be prepared ready for execution.

Object execute()

Object execute(Object param)

Object execute(Object param1, Object param2)

Object execute(Object param1, Object param2, Object param3)

Object executeWithArray(Object[] params)

Object executeWithMap(Map params)

Once a Query has been defined it can be executed. An `execute()` method returns an Object. If the Query language being used is JDO-QL then returned Object is an un-modifiable `java.util.Collection` that contains the persistent objects that satisfy the filter. If the Query language is not JDO-QL then the class of the returned Object is implementation specific.

The returned `java.util.Collection` can be used to iterate through the set of persistent objects that satisfy the specified filter. `UnsupportedOperationException` will be thrown if the collection is modified in any way. The `size()` method may return `Integer.MAX_VALUE` if the number of instances is unknown, depending on the JDO implementation the `execute()` method may use a cursor or some kind and therefore the actual number of instances in the result set is not known until they have all been retrieved.

If the Query does not have any parameters then the `execute()` method with no parameters can be used.

If the Query defines up to three parameters then one of the `execute()` methods that takes one, two or three Object parameters can be used. The Object parameters are bound in order to the parameters defined by the `declareParameters()` method (the first parameter defined is bound to the first value specified in the `execute()` method and so on). The types of the parameters specified in the `execute()` method should match those specified by the `declareParameters()` method, otherwise

JDOUserException is thrown. For primitive types an instance of the equivalent wrapper class should be used.

If more than three parameters are defined then the `executeWithArray()` or `executeWithMap()` methods should be used.

The `executeWithArray()` method takes an array of parameters and binds each one in order to the parameters defined by the `declareParameters()` method.

The `executeWithMap()` method takes a map of parameters. Each key should be a string that matches the name of a parameter defined by the `declareParameters()` method. The associated value will be bound to the matching parameter in the Query.

If a value for a parameter is not specified when a Query is executed JDOUserException is thrown.

Closing Results

It is best practice to ensure that any result returned from an `execute()` method is closed when finished with. Depending on the JDO implementation this may release datastore resources used during the execution of the Query (a cursor for example).

5.3.4.4 Usage Guidelines

Chapter X on provides greater details on the JDO-QL syntax and how to use queries within a JDO application. The following examples serve to provide an introduction to the basics.

To find a persistent object by a field value requires only a simple query. The following code snippet taken from “SimpleQueryExample.java” shows how to create and execute a simple query to find all instances of Book with a given title. The Book’s title is hard coded into the filter string used to create the Query:

```
Query query =
    pm.newQuery(
        Book.class,
        "title == \"Core Java Data Objects\"");

Collection result = (Collection) query.execute();

Iterator iter = result.iterator();

while (iter.hasNext()) {
```

```
Book book = (Book) iter.next();

System.out.println(
    book.getTitle() + " - " + book.getISBN());
}

query.close(result);
```

Rather than hard coding a literal in the filter string a parameter can be used instead. The following code snippet taken from “SimpleQueryWithParameterExample.java” is the same as the previous example except it defines a parameter to specify the Book’s title rather than hard coding it. The “this” keyword is used to resolve the ambiguity between the class’ field name and the parameter’s name (both of which are called “title”):

```
Query query =
    pm.newQuery(Book.class, "this.title == title");

query.declareParameters("String title");

Collection result = (Collection)
    query.execute("Core Java Data Objects");

Iterator iter = result.iterator();

while (iter.hasNext()) {

    Book book = (Book) iter.next();

    System.out.println(
        book.getTitle() + " - " + book.getISBN());
}

query.close(result);
```

The results from a query are not returned in any particular order by default. To specify an ordering use the `setOrdering()` method. The following code snippet taken from “SimpleQueryWithOrderingExample.java” extends the previous example but specifies that the returned Book instances should be ordered by their publication dates:

```
Query query =
    pm.newQuery(Book.class, "this.title == title");

query.declareParameters("String title");

query.setOrdering("publicationDate ascending");
```

```

Collection result = (Collection)
    query.execute("Core Java Data Objects");

Iterator iter = result.iterator();

while (iter.hasNext()) {

    Book book = (Book) iter.next();

    System.out.println(
        book.getTitle() + " - " + book.getISBN());
}

query.close(result);

```

A Query can also use navigation to specify fields of referenced instances. The following code snippet taken from “QueryWithNavigationExample.java” shows how to use the “.” notation to navigate to fields of a referenced instance. The query returns all books published by a given publisher using the “.” notation to check the name field of the Publisher instance referenced by the “publisher” field for each book:

```

Query query =
    pm.newQuery(Book.class, "publisher.name == name");

query.declareParameters("String name");

Collection result = (Collection)
    query.execute("Sun Microsystems Press");

Iterator iter = result.iterator();

while (iter.hasNext()) {

    Book book = (Book) iter.next();

    System.out.println(
        book.getTitle() + " - " + book.getISBN());
}

query.close(result);

```

A Query can also navigate to multiple persistent objects contained in a collection and check the fields of those contained persistent objects. If there is at least one persistent object in the collection that matches then the referencing persistent object is returned. The follow-

ing code snippet taken from “SimpleQueryUsingContainsExample.java” shows how to use “contains()” and a variable to navigate to fields of persistent objects contained in a collection. The query returns all Books by a given Author. Essentially, for each Book, the Authors contained in the “authors” collection are checked to see if their name matches that given. If there is at least one Author contained in the collection with the given name then the Book is returned:

```
Query query =
    pm.newQuery(
        Book.class,
        "authors.contains(author) && author.name == name");

query.declareParameters("String name");
query.declareVariables("Author author");

Collection result = (Collection)
    query.execute("Keiron McCammon");

Iterator iter = result.iterator();

while (iter.hasNext()) {

    Book book = (Book) iter.next();

    System.out.println(
        book.getTitle() + " - " + book.getISBN());
}

query.close(result);
```

5.3.5 Transaction

The Transaction interface provides the methods that allow an application to control the underlying datastore transactions. Each PersistenceManager has an associated Transaction instance. A Transaction is initially inactive, becomes active after begin() and remains active until either commit() or rollback(), at which point it becomes inactive again.

5.3.5.1 Creating a Transaction

The currentTransaction() method defined on PersistenceManager can be used to get the Transaction instance associated with a PersistenceManager, each PersistenceManager has one Transaction instance:

```
public Transaction currentTransaction()
```

5.3.5.2 Transaction Properties

Transaction defines a number of properties that control various semantics of a transaction. Once set the properties of a Transaction remain so until changed again (they are not affected by transaction completion).

Support for a number of the Transaction properties is optional, if an implementation does not support a particular optional feature then `JDOUnsupportedOperationException` will be thrown if the property is set to true.

Table 5-7 Transaction Properties

Property	Method Summary
NontransactionalRead*	boolean <code>getNontransactionalRead()</code> void <code>setNontransactionalRead(boolean flag)</code> This property determines if non-transactional reads are allowed. If true they are allowed, if false an attempt to access a persistent object outside of a transaction will result in <code>JDOUserException</code> being thrown. If non-transactional read is not supported by a JDO implementation <code>JDOUnsupportedOption</code> will be thrown if an attempt is made to set this property to true.
NontransactionalWrite*	boolean <code>getNontransactionalWrite()</code> void <code>setNontransactionalWrite(boolean flag)</code> This property determines if non-transactional writes are allowed. If true they are allowed, if false an attempt to modify a persistent object outside of a transaction will result in <code>JDOUserException</code> being thrown. If non-transactional write is not supported by a JDO implementation <code>JDOUnsupportedOption</code> will be thrown if an attempt is made to set this property to true.
Optimistic*	boolean <code>getOptimistic()</code> void <code>setOptimistic(boolean flag)</code> This property determines if the transaction is optimistic or not. True indicates the Transaction is optimistic. This property cannot be changed if a transaction is currently active, <code>JDOUserException</code> will be thrown. If optimistic transactions are not supported by a JDO implementation <code>JDOUnsupportedOption</code> will be thrown if an attempt is made to set this property to true.
PersistenceManager	<code>PersistenceManager</code> <code>getPersistenceManager()</code> This property denotes the <code>PersistenceManager</code> associated with the Transaction instance.
RestoreValues*	boolean <code>getRestoreValues()</code> void <code>setRestoreValues(boolean flag)</code> This property determines if the fields of persistent objects are restored to their values as of

Property	Method Summary
	the beginning of a transaction on rollback. If true they will be restored and the persistent objects will transition to being persistent, non-transactional at the end of the transaction. This property cannot be changed if a transaction is currently active, <code>JDOUserException</code> will be thrown. If restoring values is not supported by a JDO implementation <code>JDOUnsupportedOption</code> will be thrown if an attempt is made to set this property to true.
RetainValues*	<code>boolean getRetainValues()</code> <code>void setRetainValues(boolean flag)</code> This property determines if the field values of persistent objects should be retained in-memory after a transaction ends. If true they will be retained any in-memory persistent objects will transition to being persistent, non-transactional at the end of the transaction. If retaining values is not supported by a JDO implementation <code>JDOUnsupportedOption</code> will be thrown if an attempt is made to set this property to true.
Synchronization	<code>Object getSynchronization()</code> <code>void setSynchronization(Synchronization s)</code> This property allows an application to register an instance of the <code>Synchronization</code> interface with the <code>Transaction</code>. If not null then the <code>beforeCompletion()</code> and <code>afterCompletion()</code> methods of the specified <code>Synchronization</code> instance will be invoked before and after <code>commit()</code> and <code>afterCompletion()</code> will be invoked after <code>rollback()</code>. If no <code>Synchronization</code> callback is required then null can be used.

*Support is optional.

5.3.5.3 Method Summary

The `Transaction` methods allow an application to explicitly control transactions. If used within a managed environment where transactions are controlled by an external coordinator then any attempt to explicitly begin or end a transaction will result in `JDOUserException` being thrown. Only `isActive()` can be called when using an external coordinator.

void begin()

This method can be used to begin a transaction.

If called when an external coordinator is being used
`JDOUserException` is thrown.

void commit()

This method can be used to end a transaction and commit all new, modified and deleted instances to the database.

If called when an external coordinator is being used
`JDOUserException` is thrown.

boolean isActive()

This method returns true if the Transaction is active (i.e. begin() has been called, but no commit() or rollback() has yet occurred), false otherwise.

void rollback()

This method can be used to rollback a transaction and discard all changes (new, modified or deleted instances).

If called when an external coordinator is being used JDOUserException is thrown.

5.3.5.4 Usage Guidelines

Using the Transaction interface is simple. The following code snippet taken from “TransactionExample.java” shows how to begin and commit a transaction:

```
PersistenceManager pm = pmf.getPersistenceManager();

Transaction tx = pm.currentTransaction();

tx.begin();

// Do something interesting

tx.commit();
```

The following code snippet taken from “NonTransactionalReadExample.java” shows how a persistent object can be accessed outside of a transaction using a non-transactional read. If setNontransactionalRead(true) was not supported then accessing a persistent object outside of a transaction would result in JDOUserException being thrown:

```
Transaction tx = pm.currentTransaction();

tx.setNontransactionalRead(true);

tx.begin();

Author author =
    new Author(
        "Keiron McCammon",
        new Address(
            "6539 Dumbarton Circle", "Fremont", "CA", "94555"));
```

```
pm.makePersistent(author);

tx.commit();

// Author instance transitions to hollow after commit

System.out.println(
    "Author's name is '" + author.getName() + "'");

// Author instance is retrieved from the datastore and
// transitions to being non-transactional

if (!JDOHelper.isTransactional(author)) {

    System.out.println("Author is non-transactional.");
}
```

The output would be:

```
Author's name is 'Keiron McCammon'.
Author is non-transactional.
```

The following code snippet taken from “RetainValuesExample.java” differs from the previous example only in that it calls `setRetainValues()` instead of `setNontransactionalRead()`. In the previous example the Author instance transitions to being hollow after the commit and when accessed outside of the transaction its fields are re-retrieved from the datastore and the instance transitions to being non-transactional. With `retainValues()` the Author instance transitions to being non-transactional after the commit and its field values are kept in-memory. This has the advantage that the fields of the instance don’t have to be re-retrieved again:

```
Transaction tx = pm.currentTransaction();

tx.setRetainValues(true);

tx.begin();

Author author =
    new Author(
        "Keiron McCammon",
        new Address(
            "6539 Dumbarton Circle", "Fremont", "CA", "94555"));

pm.makePersistent(author);
```

```

tx.commit();

// Author instance transitions to being non-transactional
// after commit and its fields remain in-memory

if (!JDOHelper.isTransactional(author)) {

    System.out.println("Author is non-transactional.");
}

System.out.println(
    "Author's name is '" + author.getName() + "'");

```

The output would be:

```

Author is non-transactional.
Author's name is 'Keiron McCammon'.

```

Retain values can be used to retain the fields of persistent objects in-memory after a transaction commits. However, if a rollback is used instead of a commit then the in-memory instances will retain the values of any fields that were modified during the transaction, even though the modifications will be rolled back in the datastore. If this is undesirable then the `restore values` option can be used to revert the modified fields to the values they had at the beginning of the transaction. The following code snippet taken from `RestoreValuesExample.java` shows that if the name of an `Author` instance is modified but the transaction is rolled back then the modified name remains in-memory unless `restore values` is used:

```

Transaction tx = pm.currentTransaction();

tx.setRetainValues(true);

tx.begin();

Author author =
    new Author(
        "Keiron McCammon",
        new Address(
            "6539 Dumbarton Circle", "Fremont", "CA", "94555"));

pm.makePersistent(author);

tx.commit();

// Author instance is retained after commit and transitions

```

```
// to being non-transactional
if (!JDOHelper.isTransactional(author)) {
    System.out.println("Author is non-transactional.");
}

System.out.println(
    "Author's name is '" + author.getName() + "'");

tx.begin();
author.setName("Sameer Tyagi");
tx.rollback();

// Transaction was rolled back rather than committed
// therefore the name change is not in the datastore.
// However, the changed name is retained in-memory.

System.out.println(
    "Author's name is '" + author.getName() +
    "' after rollback.");

tx.setRestoreValues(true);
tx.begin();
author.setName("Sameer Tyagi");
tx.rollback();

// This time because RestoreValues is true the author's
// name is reverted back to its original name on rollback

System.out.println(
    "Author's name is '" + author.getName() +
    "' after rollback with RestoreValues.");

The output would be:

Author is non-transactional.
Author's name is 'Keiron McCammon'.
Author's name is 'Sameer Tyagi' after rollback.
```

Author's name is 'Keiron McCammon' after rollback with `RestoreValues`.

The following code snippet taken from "OptimisticExample.java" changes the zipcode of an Author's address in an optimistic transaction and makes the Author instance transactional. In another transaction the name of the Author is modified and committed before the original optimistic transaction is committed. When the optimistic transaction is committed an exception will be thrown because the Author instance was modified by the other transaction. If the Author instance hadn't been explicitly made transactional the commit would have been successful:

```
Transaction tx = pm.currentTransaction();

tx.setOptimistic(true);

tx.begin();

Author author = Author.findByName(pm, "Keiron McCammon");
Address address = author.getAddress();
address.setZipcode("12345");
pm.makeTransactional(author);

// Author's name is modified in another transaction
PersistenceManager pm2 = pmf.getPersistenceManager();
Transaction tx2 = pm2.currentTransaction();

tx2.begin();

Author author2 = Author.findByName(pm2, "Keiron McCammon");
author2.setName("Sameer Tyagi");
tx2.commit();

pm2.close();

// Because the Author's name was updated in a different
// transaction the optimistic transaction will throw
// an exception on commit.
```

```
try {  
    tx.commit();  
}  
  
catch (JDOUserException e) {  
    tx.rollback();  
}
```

5.3.6 InstanceCallbacks

The InstanceCallbacks interface provides a means whereby instances of persistence capable classes can take action on certain state changes in a persistent object's lifecycle. The JDO implementation invokes these methods as persistent objects are stored, retrieved and deleted.

5.3.6.1 Method Summary

Figure 5-2 provides a simplified view of when the various InstanceCallbacks methods are invoked:

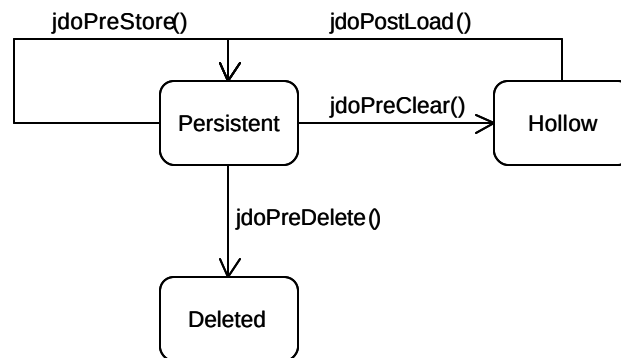


Figure 5-2 Invocation of InstanceCallbacks

When a persistent object is retrieved from the datastore `jdoPostLoad()` is invoked. Before a persistent object is sent to the datastore (typically during commit) `jdoPreStore()` is invoked. If a persistent object is deleted then `jdoPreDelete()` is invoked and before a persistent object transitions to being hollow `jdoPreClear()` is invoked.

void jdoPostLoad()

This method is invoked after the fields in the default fetch group have been retrieved from the datastore.

Access to fields in the default fetch group is permitted but other fields will be un-initialized and should not be accessed. Access to other persistent objects from within this method is also not allowed. Any exceptions thrown during this method are ignored.

This method is useful to initialize the values of any non-persistent fields or to register the in-memory persistent object with other instances in the JVM.

void jdoPreClear()

This method is invoked before the fields of an instance are cleared. The fields are cleared when a persistent object transitions to being hollow, typically at the end of a transaction or the result of an eviction.

Access to fields in the default fetch group is permitted but other fields may not have been initialized and should not be accessed. Any exceptions thrown during this method are ignored.

This method is useful to clear the values of any non-persistent, non-transactional fields or to un-register the in-memory persistent object with other instances in the JVM.

void jdoPreDelete()

This method is invoked before an instance is deleted. An instance is deleted using the `deletePersistent()` method on `PersistenceManager`.

Access to all fields is permitted within this method. Any exceptions thrown during this method will be propagated to the application immediately and the persistent object won't be deleted.

This method is useful to cascade delete other referenced instances.

void jdoPreStore()

This method is invoked before the fields of an instance are sent to the datastore. This typically happens during commit.

Access to all fields is permitted within this method. Any exceptions thrown during this method will be propagated to the application immediately and the persistent object won't be stored in the datastore.

This method is useful to update persistent fields based on the values of non-persistent fields or implement simple constraint checks on field values.

5.3.6.2 Usage Guidelines

The following code snippet taken from "Author.java" shows how `jdoPreDelete()` can be used to delete an Author's Address and remove the Author from each of its referenced books:

```
public class Author implements InstanceCallbacks {

    private String name;
    private Address address;
    private Set books;

    // Additional methods not shown

    public void jdoPostLoad() {
    }

    public void jdoPreClear() {
    }

    public void jdoPreDelete() {

        JDOHelper.getPersistenceManager(this).
            deletePersistent(address);

        if (books != null) {

            Iterator iter = books.iterator();

            while (iter.hasNext()) {

                Book book = (Book) iter.next();
```

```

        book.removeAuthor(this);
    }
}

public void jdoPreStore() {
}
}

```

5.4 EXCEPTION CLASSES

JDO defines a set of exception classes that a JDO runtime can throw. The exceptions are all runtime exceptions since they can occur whenever an in-memory persistent object is accessed, not just when a JDO method is invoked. The classes are divided into fatal and non-fatal exceptions. Refer to figure x-x for the JDO exception class hierarchy.

A JDO exception behaves the same as a normal Java runtime exception, however in addition to an error string a JDO exception may also have a reference to the persistent object that the exception relates to. For operations that process more than one persistent object (a commit for example) a JDO exception will contain an array of nested exceptions, one for each persistent object that encountered a problem.

5.4.1 JDOException

This is the base class for all JDO exceptions and defines the methods common to all exception classes. In addition the `toString()` and `printStackTrace()` methods have been overridden to include details on any nested exceptions also.

5.4.1.1 Method Summary

An application can get a reference to the persistent object that the exception relates to and an array of nested exceptions where multiple problems were encountered.

Object `getFailedObject()`

This method returns the persistent object that the exception relates to or null if there is no specific persistent object.

Throwable[] `getNestedExceptions()`

This method returns an array of nested exceptions where multiple problems were encountered, or null if there are no nested exceptions.

5.4.2 JDOFatalException

This is the base class for fatal exceptions. An operation that resulted in a fatal exception cannot be re-tried. Typically the only recourse is to restart a transaction or even get a new `PersistenceManager` instance.

5.4.3 JDOFatalUserException

This exception indicates a fatal problem with the application. It typically means that the application has called a method at an inappropriate time. Cases where this exception might be thrown are:

- A method, other than `isClosed()`, is called on `PersistenceManager` after it has been closed or instances associated with a closed `PersistenceManager` are accessed (Query, Extent, Transaction, and soon).
- The `getPersistenceManagerFactory()` method on `JDOHelper` cannot find the specified `PersistenceManagerFactory` class or unsupported options have been specified.

5.4.4 JDOFatalInternalException

This exception indicates a fatal internal problem with the JDO implementation. There is little recourse other than to report the problem to the vendor for rectification.

5.4.5 JDOFatalDataStoreException

This exception indicates a fatal problem with the datastore. Any active transaction will have been rolled back. Cases where this exception might be thrown are:

- The current transaction has been automatically rolled back by the datastore, perhaps due to a timeout.
- The connection to the datastore has been lost, perhaps due to a network problem.

5.4.6 JDOCanRetryException

This is the base class for non-fatal exceptions. An operation that resulted in a non-fatal exception can be retried.

5.4.7 JDOUnsupportedOptionException

This exception indicates a non-fatal application problem. The application has tried to use an optional feature that the JDO implementation does not support. Cases where this exception might be thrown are:

- Invoking `setOptimistic()` on `Transaction` when optimistic transactions aren't supported by the JDO implementation
- Trying to use collection classes not supported by the JDO implementation.

5.6.8 JDOUserException

This exception indicates a non-fatal application problem. The application can continue and potentially rectify the problem and retry the operation. Cases where this exception might be thrown are:

- An instance of a non-persistence capable class is passed to a method that expects an instance of a persistence capable class.
- The `getExtent()` method is used on `PersistenceManager` when the class does not have a managed extent.
- A persistent object that no longer exists in the datastore is accessed.

5.4.9 JDODataStoreException

This exception indicates a non-fatal problem with the datastore. The application can continue and potentially rectify the problem and retry the operation.

5.4.10 JDOObjectNotFoundException

This exception indicates a non-fatal application problem. Essentially a persistent object could not be found in the datastore. Cases where this exception might be thrown are;

- Dereferencing a reference to a persistent object that has previously been deleted, perhaps by a different application.
- The `getObjectById()` method on `PersistenceManager` is unable to find a persistent object with the specified object identity.

5.5 ADDITIONAL APIs

JDO defines two additional classes that are usable by an application:

- `JDOHelper`
- `I18NHelper`

5.5.1 JDOHelper Class

This class defines a number of static methods that a persistence capable class or JDO application can use. Each method simply delegates to other JDO classes, like `PersistenceManager` and `PersistenceManagerFactory` as well as classes defined in the “`javax.jdo.spi`” package like `PersistenceCapable`.

5.5.1.1 Method Summary

This class defines methods that can be used as a bootstrap mechanism for creating a `PersistenceManagerFactory` instance as well as convenience methods for interacting with persistent objects.

static `PersistenceManager` `getPersistenceManager(Object pc)`

This method returns the `PersistenceManager` instance associated with the specified object. It returns null if the specified instance is transient or is not an instance of a persistence capable class.

static `PersistenceManagerFactory`

`getPersistenceManagerFactory(Properties props)`

static `PersistenceManagerFactory`

`getPersistenceManagerFactory(Properties props, ClassLoader cl)`

These methods create a `PersistenceManagerFactory` instance based on the specified properties. See [section X.X](#) for more details.

If no class loader is specified then the calling thread's context `ClassLoader` is used to find the `PersistenceManagerFactory` class.

static Object getId(Object pc)

static Object getTransactionalId(Object pc)

These methods provide short-cuts for the `getId()` and `getTransactionalId()` methods defined on `PersistenceManager`.

static boolean isDirty(Object pc)

static boolean isTransactional(Object pc)

static boolean isPersistent(Object pc)

static boolean isNew(Object pc)

static boolean isDeleted(Object pc)

These methods allow an application to interrogate the lifecycle state of a persistent object and determine if it is dirty, transactional, persistent, new or deleted.

See chapter 4 for more details on the object lifecycle.

static void makeDirty(Object pc, String fieldName)

This method allows an application to explicitly mark the specified field of the given persistent object as dirty. The `fieldName` is the name of the field, optionally fully qualified with the package and class name. If the specified object is null, transient or not an instance of a persistence capable class or the specified field name is not a managed field then this method does nothing.

5.5.2 I18NHelper Class

This class provides localized strings for the classes in the “`javax.jdo`” and “`javax.jdo.spi`” packages (primarily the exception classes). It finds a `Java ResourceBundle` for the current locale using “`java.jdo.Bundle`” as the baseclass. If no `ResourceBundle` is

found for the current locale then the “Bundle.properties” file located in the “javax.jdo” package is used.

A JDO application doesn’t need to call any methods defined on this class, only if it needs to provide localized strings would it need to provide its own ResourceBundle.

5.6 SERVICE PROVIDER INTERFACE APIs

In addition to the application visible classes contained in the “javax.jdo” package there are a number of classes in the “javax.jdo.spi” package. These classes are used by the JDO implementations themselves and an application should not use these classes directly. A brief summary of the following classes is provided as background information:

- PersistenceCapable
- JDOPermission
- JDOImplHelper
- StateManager

5.6.1 PersistenceCapable

This is the interface that all persistence capable classes implement. It defines the methods that a JDO implementation uses to manage the instances of persistence capable classes in-memory. It primarily has methods to get and set the fields of an instance and to interrogate lifecycle state. The implementation of the methods defined by PersistenceCapable delegate to a StateManager instance to actually manage the fields.

An application should use the methods defined on PersistenceManager and JDOHelper to interact with persistent objects in-memory.

5.6.2 JDOPermission

JDO defines two privileged operations that a JDO implementation requires:

- setStateManager – allows an implementation to set the StateManager instance associated with an instance of a persistence capable class and therefore access the fields of that instance.
- getMetaData – allows an implementation to access the metadata for a persistence capable class via JDOImplHelper.

The JDOPermission class is used to determine what can perform a privileged operation. If a JDO application is run using a security manager (perhaps as an applet) then the classes that

constitute the JDO implementation need to be granted the “setStateManager” and “get-MetaData” privileges in a policy file.

5.6.3 JDOImplHelper

This class is a singleton that maintains a registry of metadata for each persistence capable class. When a persistence capable class is loaded it registers itself with JDOImplHelper and the JDO implementation can then use the metadata directly and avoid using reflection.

5.6.4 StateManager

The StateManager interface defines how a JDO implementation actually interacts with instances of persistence capable classes and manages the fields. The PersistenceCapable and StateManager interfaces are very closely related.

5.7 SUMMARY

From reading this chapter you should now have a thorough understanding of the concepts underlying JDO and all the JDO classes and interfaces along with how to use them.

Now you know how to use JDO to build an application the next chapter will explore the different types of application architectures that can be used with JDO.